

Projet de Gestion de Borne de Recharge de Véhicule Électrique en C++

1.0

Généré par Doxygen 1.9.1

1	Projet de Gestion de Borne de Recharge de Véhicule Électrique en C++	1
1.1	Auteurs	1
1.2	Description	1
1.3	Machine à état du Système	1
1.4	Diagramme de Classe	1
2	Machine à Etat	3
2.1	Machine à états	3
3	Diagramme de classes	5
3.1	Diagramme de classes	5
4	Index des classes	7
4.1	Liste des classes	7
5	Index des fichiers	9
5.1	Liste des fichiers	9
6	Documentation des classes	11
6.1	Référence de la classe Baseclient	11
6.1.1	Description détaillée	11
6.1.2	Documentation des constructeurs et destructeur	12
6.1.2.1	Baseclient()	12
6.1.3	Documentation des fonctions membres	12
6.1.3.1	baseclient_afficher_clients()	13
6.1.3.2	baseclient_ajoutclient()	13
6.1.3.3	baseclient_authentifier()	14
6.1.3.4	baseclient_modifierclient()	15
6.1.3.5	baseclient_supprimeclient()	16
6.1.3.6	charger()	17
6.1.3.7	sauvegarder()	18
6.1.4	Documentation des données membres	18
6.1.4.1	clients	18
6.1.4.2	fichier	18
6.2	Référence de la classe Bouton	19
6.2.1	Description détaillée	19
6.2.2	Documentation des constructeurs et destructeur	19
6.2.2.1	Bouton()	19
6.2.3	Documentation des fonctions membres	20
6.2.3.1	bouton_appuie_button_stop()	20
6.2.3.2	bouton_appuie_button_charge()	21
6.2.3.3	bouton_set_bouton_charge()	21
6.2.3.4	bouton_set_bouton_stop()	22
6.2.4	Documentation des données membres	23

6.2.4.1 io_b	23
6.3 Référence de la structure Client	23
6.3.1 Description détaillée	23
6.3.2 Documentation des données membres	24
6.3.2.1 nom	24
6.3.2.2 numero	24
6.4 Référence de la classe Generateur_Save	24
6.4.1 Description détaillée	24
6.4.2 Documentation des constructeurs et destructeur	25
6.4.2.1 Generateur_Save()	25
6.4.3 Documentation des fonctions membres	25
6.4.3.1 generateur_save_fermer_contacteur()	25
6.4.3.2 generateur_save_generer_pwm()	26
6.4.3.3 generateur_save_ouvrir_contacteur()	27
6.4.3.4 generateur_save_tension_DC()	28
6.4.4 Documentation des données membres	29
6.4.4.1 io_g	29
6.5 Référence de la classe LecteurCarte	29
6.5.1 Description détaillée	29
6.5.2 Documentation des fonctions membres	30
6.5.2.1 lecteurcarte_initialiser()	30
6.5.2.2 lecteurcarte_initialiser_lecteur()	30
6.5.2.3 lecteurcarte_lire_carte()	31
6.6 Référence de la classe Prise	32
6.6.1 Description détaillée	32
6.6.2 Documentation des constructeurs et destructeur	33
6.6.2.1 Prise()	33
6.6.3 Documentation des fonctions membres	33
6.6.3.1 prise_deverrouille_trappe()	34
6.6.3.2 prise_set_prise()	34
6.6.3.3 prise_verrouille_trappe()	35
6.6.4 Documentation des données membres	36
6.6.4.1 io_p	36
6.7 Référence de la classe Timer	36
6.7.1 Description détaillée	37
6.7.2 Documentation des constructeurs et destructeur	37
6.7.2.1 Timer()	37
6.7.3 Documentation des fonctions membres	37
6.7.3.1 timer_count_sec()	38
6.7.3.2 timer_raz()	39
6.7.3.3 timer_valeur()	39
6.7.4 Documentation des données membres	40

6.7.4.1 depart_timer	40
6.7.4.2 io_t	40
6.8 Référence de la classe Voyant	41
6.8.1 Description détaillée	41
6.8.2 Documentation des constructeurs et destructeur	41
6.8.2.1 Voyant()	42
6.8.3 Documentation des fonctions membres	42
6.8.3.1 voyant_blink_charge()	42
6.8.3.2 voyant_blink_default()	43
6.8.3.3 voyant_set_charge()	44
6.8.3.4 voyant_set_default()	45
6.8.3.5 voyant_set_dispo()	46
6.8.3.6 voyant_set_prise()	47
6.8.3.7 voyant_set_trappe()	47
6.8.4 Documentation des données membres	48
6.8.4.1 io	48
7 Documentation des fichiers	49
7.1 Référence du fichier baseclient.cpp	49
7.1.1 Description détaillée	49
7.2 Référence du fichier baseclient.h	50
7.2.1 Description détaillée	51
7.2.2 Documentation des macros	51
7.2.2.1 IOSTREAM	51
7.3 Référence du fichier borne.cpp	51
7.3.1 Description détaillée	52
7.3.2 Documentation du type de l'énumération	52
7.3.2.1 etatsystem	52
7.3.3 Documentation des fonctions	53
7.3.3.1 administration_operateur()	53
7.3.3.2 main()	54
7.3.4 Documentation des variables	56
7.3.4.1 baseclient	56
7.3.4.2 lecteurcarte	57
7.4 Référence du fichier bouton.cpp	57
7.4.1 Description détaillée	57
7.5 Référence du fichier bouton.h	57
7.5.1 Description détaillée	58
7.5.2 Documentation des macros	59
7.5.2.1 IOSTREAM	59
7.6 Référence du fichier generateur_save.cpp	59
7.6.1 Description détaillée	59

7.7 Référence du fichier <code>generateur_save.h</code>	60
7.7.1 Description détaillée	61
7.7.2 Documentation des macros	61
7.7.2.1 IOSTREAM	61
7.8 Référence du fichier <code>lecteurcarte.cpp</code>	61
7.8.1 Description détaillée	62
7.9 Référence du fichier <code>lecteurcarte.h</code>	62
7.9.1 Description détaillée	63
7.9.2 Documentation des macros	63
7.9.2.1 IOSTREAM	63
7.10 Référence du fichier <code>page2.md</code>	63
7.11 Référence du fichier <code>page3.md</code>	63
7.12 Référence du fichier <code>prise.cpp</code>	63
7.12.1 Description détaillée	64
7.13 Référence du fichier <code>prise.h</code>	64
7.13.1 Description détaillée	65
7.13.2 Documentation des macros	65
7.13.2.1 IOSTREAM	65
7.14 Référence du fichier <code>README.md</code>	65
7.15 Référence du fichier <code>timer.cpp</code>	65
7.15.1 Description détaillée	66
7.16 Référence du fichier <code>timer.h</code>	66
7.16.1 Description détaillée	67
7.16.2 Documentation des macros	67
7.16.2.1 IOSTREAM	67
7.17 Référence du fichier <code>voyant.cpp</code>	67
7.17.1 Description détaillée	68
7.18 Référence du fichier <code>voyant.h</code>	68
7.18.1 Description détaillée	69
7.18.2 Documentation des macros	69
7.18.2.1 IOSTREAM	69
Index	71

Chapitre 1

Projet de Gestion de Borne de Recharge de Véhicule Électrique en C++

1.1 Auteurs

- KAMDA TEZEBO DIBREY JONATAN
- Nassime BOUANANI

1.2 Description

Système de gestion de borne de recharge pour véhicule électrique.

1.3 Machine à état du Système

Voir ici : [Machine à Etat](#)

1.4 Diagramme de Classe

Voir le diagramme de classes : [Diagramme de classes](#)

Chapitre 2

Machine à Etat

2.1 Machine à états

Machine à état du système de gestion de la borne de recharge :

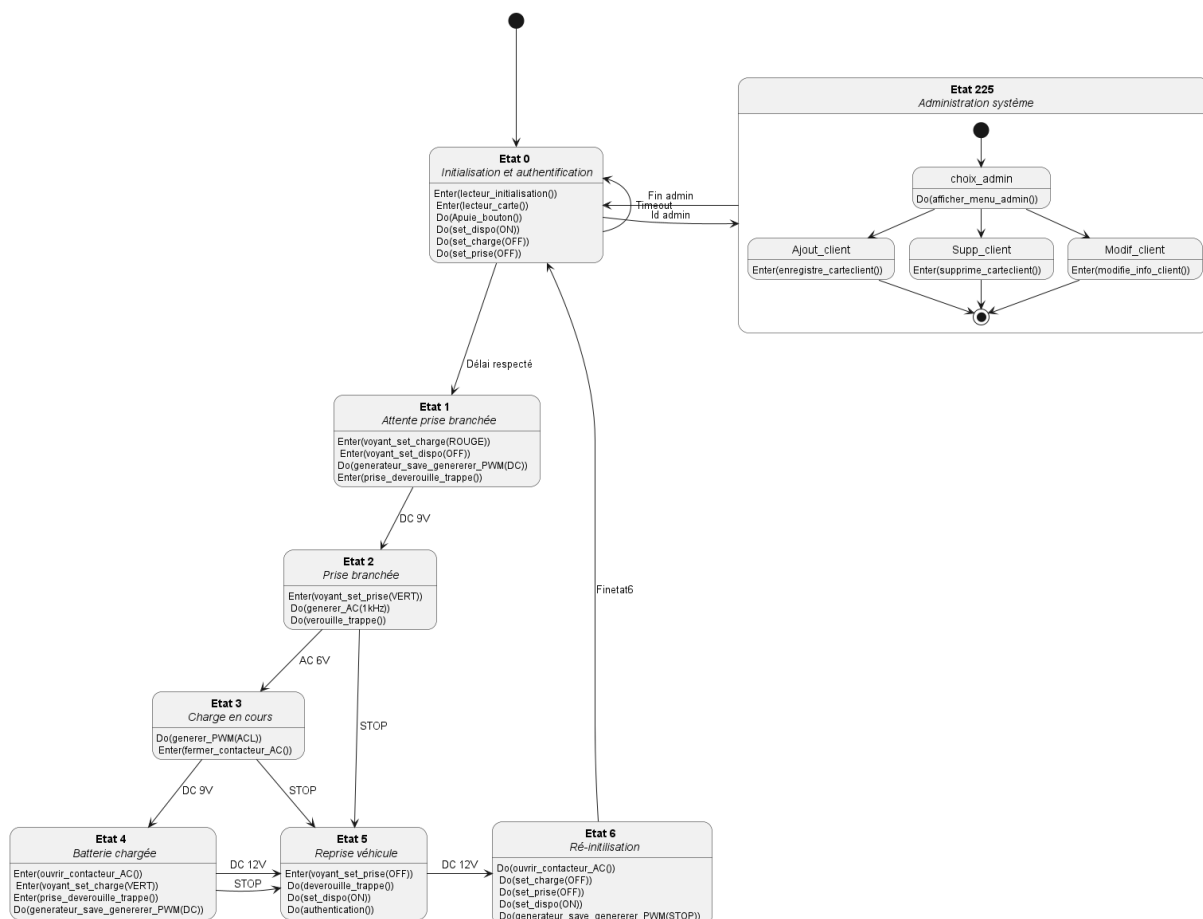


FIGURE 2.1 Machine à états

Chapitre 3

Diagramme de classes

3.1 Diagramme de classes

Voici le diagramme de classes représentant l'architecture logicielle de la borne de recharge :

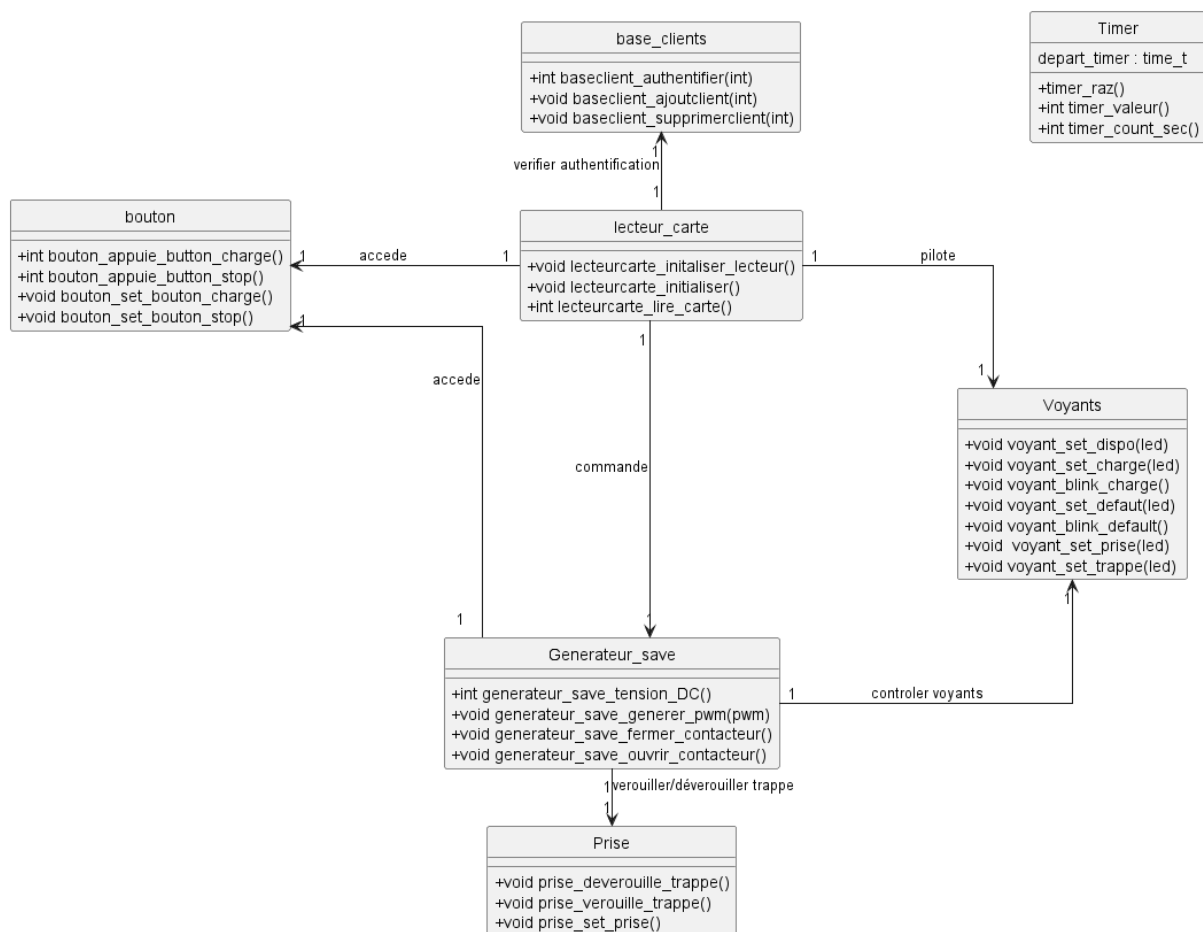


FIGURE 3.1 Diagramme de classes

Chapitre 4

Index des classes

4.1 Liste des classes

Liste des classes, structures, unions et interfaces avec une brève description :

Baseclient	Classe de gestion de la base de données clients de la borne	11
Bouton	Gère la lecture et la remise à zéro des boutons physiques de la borne	19
Client	Représente un client enregistré dans la base de données	23
Generateur_Save	Pilote le générateur PWM, le contacteur AC et la lecture de tension DC de la borne de recharge	24
LecteurCarte	Gère l'initialisation et la lecture du lecteur de cartes	29
Prise	Pilote la trappe mécanique et le voyant de la prise de charge	32
Timer	Gère la temporisation et la mesure du temps écoulé dans la borne	36
Voyant	Pilote l'ensemble des voyants lumineux de la borne de recharge	41

Chapitre 5

Index des fichiers

5.1 Liste des fichiers

Liste de tous les fichiers avec une brève description :

baseclient.cpp	Implémentation des méthodes de la classe Baseclient	49
baseclient.h	Déclaration de la structure Client et de la classe Baseclient	50
borne.cpp	Fichier principal du système de borne de recharge Ce fichier contient les objets de chaque classe	51
bouton.cpp	Implémentation des méthodes de la classe Bouton	57
bouton.h	Déclaration de la classe Bouton — gestion des boutons physiques de la borne	57
generateur_save.cpp	Implémentation des méthodes de la classe Generateur_Save	59
generateur_save.h	Déclaration de la classe Generateur_Save — pilotage du générateur de signal PWM et du contacteur AC de la borne de recharge	60
lecteurcarte.cpp	Implémentation des méthodes de la classe LecteurCarte	61
lecteurcarte.h	Déclaration de la classe LecteurCarte — gestion du lecteur de cartes de la borne de recharge	62
prise.cpp	Implémentation des méthodes de la classe Prise	63
prise.h	Déclaration de la classe Prise — gestion de la trappe et du voyant de la prise de charge de la borne	64
timer.cpp	Implémentation des méthodes de la classe Timer	65
timer.h	Déclaration de la classe Timer — gestion de la temporisation du système de borne de recharge	66
voyant.cpp	Implémentation des méthodes de la classe Voyant	67
voyant.h	Déclaration de la classe Voyant — gestion des voyants lumineux de la borne de recharge . . .	68

Chapitre 6

Documentation des classes

6.1 Référence de la classe Baseclient

Classe de gestion de la base de données clients de la borne.

```
#include <baseclient.h>
```

Fonctions membres publiques

- [Baseclient](#) ()
Constructeur de la classe [Baseclient](#).
- int [baseclient_authentifier](#) (int num)
Authentifie un client à partir de son numéro de carte.
- void [baseclient_ajoutclient](#) (int num, string nom)
Ajoute un nouveau client dans la base de données.
- void [baseclient_supprimeclient](#) (int num)
Supprime un client de la base de données à partir de son numéro de carte.
- void [baseclient_afficher_clients](#) ()
Affiche la liste complète des clients enregistrés dans la base.
- void [baseclient_modifierclient](#) (int num)
Modifie le nom, le numéro de carte, ou les deux d'un client existant.

Fonctions membres privées

- void [charger](#) ()
Charge le contenu du fichier `database.txt` dans le vecteur `clients`.
- void [sauvegarder](#) ()
Sauvegarde le contenu du vecteur `clients` dans le fichier `database.txt`.

Attributs privés

- vector< [Client](#) > [clients](#)
- const string [fichier](#) = "database.txt"

6.1.1 Description détaillée

Classe de gestion de la base de données clients de la borne.

6.1.2 Documentation des constructeurs et destructeur

6.1.2.1 Baseclient()

```
Baseclient::Baseclient ( )
```

Constructeur de la classe [Baseclient](#).

À l'instanciation de l'objet, le constructeur déclenche immédiatement le chargement du fichier `database.txt` en mémoire via `charger()`. Ainsi, dès que l'objet est prêt à l'emploi, le vecteur `clients` est déjà rempli et toutes les opérations peuvent s'effectuer sans nouvel accès disque.

Charge le contenu du fichier `database.txt` dans le vecteur `clients`.

Cette méthode privée lit le fichier texte ligne par ligne en supposant un format strict de deux lignes par client :

- ligne impaire : numéro de carte (entier)
- ligne paire : nom du client (chaîne, peut contenir des espaces)

Le vecteur `clients` est vidé avant chaque chargement pour éviter les doublons en cas d'appel multiple. Si le fichier est introuvable ou illisible, un message d'erreur est affiché et la méthode retourne sans modifier le vecteur.

Note

Cette méthode est exclusivement appelée par le constructeur. Elle n'est pas destinée à être réutilisée en cours d'exécution.

Variables internes utilisées :

- **fich** : flux de lecture associé au fichier `database.txt`.
- **c** : structure [Client](#) temporaire utilisée pour construire chaque entrée avant insertion dans le vecteur.
- **ligne** : chaîne de caractères lue ligne par ligne depuis le fichier.

Voici le graphe d'appel pour cette fonction :



6.1.3 Documentation des fonctions membres

6.1.3.1 baseclient_afficher_clients()

```
void Baseclient::baseclient_afficher_clients ( )
```

Affiche la liste complète des clients enregistrés dans la base.

Parcourt le vecteur `clients` et affiche pour chaque entrée le numéro de carte et le nom du client. Si le vecteur est vide, un message l'indique et la fonction retourne immédiatement.

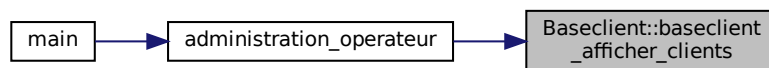
Note

Cette fonction est destinée à un usage opérateur uniquement, appelée depuis `administration_opérateur()` via le choix 3 du menu.

Renvoie

void

Voici le graphe des appelants de cette fonction :



6.1.3.2 baseclient_ajoutclient()

```
void Baseclient::baseclient_ajoutclient (
    int num,
    string nom )
```

Ajoute un nouveau client dans la base de données.

Avant tout ajout, deux vérifications sont effectuées :

- le numéro 255 est refusé car il est réservé à l'accès opérateur.
- si le numéro est déjà présent dans le vecteur, l'ajout est annulé pour éviter les doublons.

Si les vérifications sont passées, le client est ajouté en fin de vecteur puis le fichier est mis à jour via `sauvegarder()`.

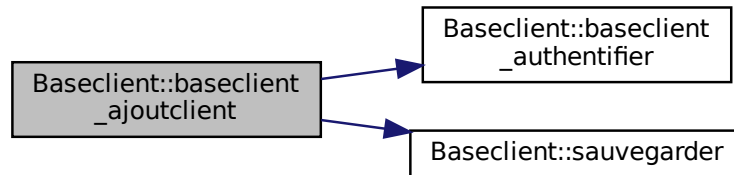
Paramètres

<i>num</i>	Numéro de carte du nouveau client. Doit être unique et différent de 255.
<i>nom</i>	Nom complet du nouveau client. Peut contenir des espaces.

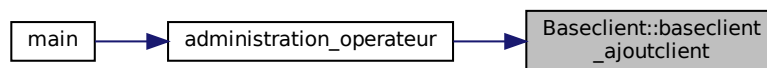
Renvoie

void

Voici le graphe d'appel pour cette fonction :



Voici le graphe des appelants de cette fonction :

**6.1.3.3 baseclient_authentifier()**

```
int Baseclient::baseclient_authentifier (
    int num )
```

Authentifie un client à partir de son numéro de carte.

Parcourt le vecteur `clients` à la recherche d'un client dont le numéro correspond au paramètre `num`. Si le client est trouvé, son nom est affiché en guise d'accueil avant de retourner 1. Si aucun client ne correspond, la fonction retourne 0 sans afficher de message.

Paramètres

<i>num</i>	Numéro de carte lu par le lecteur, à vérifier dans la base.
------------	---

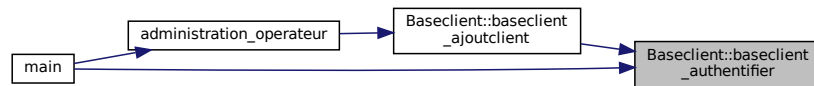
Renvoie

int

Valeurs retournées

1	si le client est reconnu dans la base de données.
0	si le numéro de carte est inconnu.

Voici le graphe des appelants de cette fonction :



6.1.3.4 baseclient_modifierclient()

```
void Baseclient::baseclient_modifierclient (
    int num )
```

Modifie le nom, le numéro de carte, ou les deux d'un client existant.

Recherche d'abord le client via son numéro `num`. Si trouvé, un sous-menu est proposé à l'opérateur avec trois options :

- **choix 1** : modification du nom uniquement.
- **choix 2** : modification du numéro de carte uniquement.
- **choix 3** : modification des deux champs.

Avant toute modification du numéro, deux garde-fous sont vérifiés :

- le numéro 255 est refusé car réservé à l'opérateur.
- le nouveau numéro ne doit pas être déjà attribué à un autre client.

Si l'une de ces conditions est violée, la modification est intégralement annulée, y compris si le nom avait déjà été modifié (choix 3). En cas de succès, le vecteur est mis à jour et persisté via `sauvegarder()`.

Paramètres

<code>num</code>	Numéro de carte du client à modifier.
------------------	---------------------------------------

Variables internes utilisées :

- **it** : itérateur pointant sur le client à modifier.
- **choix** : entier stockant le choix de modification saisi par l'opérateur.
- **nouveau_nom** : chaîne stockant le nouveau nom saisi (choix 1 et 3).
- **nouveau_num** : entier stockant le nouveau numéro saisi (choix 2 et 3).
- **doublon** : itérateur utilisé pour vérifier l'unicité du nouveau numéro.

Avertissement

Si le choix 3 est sélectionné et que le nouveau numéro est invalide, la modification du nom déjà effectuée en mémoire est perdue car `sauvegarder()` n'est pas appelée.

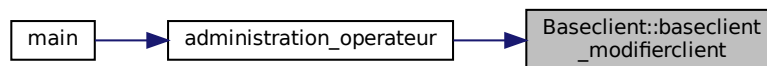
Renvoie

void

Voici le graphe d'appel pour cette fonction :



Voici le graphe des appelants de cette fonction :

**6.1.3.5 baseclient_supprimeclient()**

```
void Baseclient::baseclient_supprimeclient (
    int num )
```

Supprime un client de la base de données à partir de son numéro de carte.

Recherche dans le vecteur `clients` le premier élément dont le numéro correspond à `num` via `find_if`. Si le client est trouvé, il est retiré du vecteur avec `erase()` et le fichier est mis à jour via `sauvegarder()`. Si aucun client ne correspond, un message d'erreur est affiché et la fonction retourne sans modifier la base.

Paramètres

<i>num</i>	Numéro de carte du client à supprimer.
------------	--

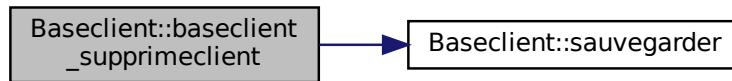
Variables internes utilisées :

- `it` : itérateur pointant sur le client trouvé dans le vecteur, ou sur `clients.end()` si introuvable.

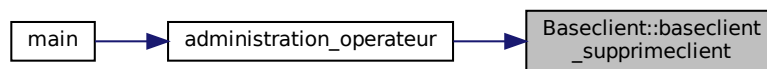
Renvoie

void

Voici le graphe d'appel pour cette fonction :



Voici le graphe des appelants de cette fonction :



6.1.3.6 charger()

```
void Baseclient::charger ( ) [private]
```

Charge le contenu du fichier `database.txt` dans le vecteur `clients`.

Cette méthode privée lit le fichier texte ligne par ligne en supposant un format strict de deux lignes par client :

- ligne impaire : numéro de carte (entier)
- ligne paire : nom du client (chaîne, peut contenir des espaces)

Le vecteur `clients` est vide avant chaque chargement pour éviter les doublons en cas d'appel multiple. Si le fichier est introuvable ou illisible, un message d'erreur est affiché et la méthode retourne sans modifier le vecteur.

Note

Cette méthode est exclusivement appelée par le constructeur. Elle n'est pas destinée à être réutilisée en cours d'exécution.

Variables internes utilisées :

- **fich** : flux de lecture associé au fichier `database.txt`.
- **c** : structure `Client` temporaire utilisée pour construire chaque entrée avant insertion dans le vecteur.
- **ligne** : chaîne de caractères lue ligne par ligne depuis le fichier.

Voici le graphe des appelants de cette fonction :



6.1.3.7 sauvegarder()

```
void Baseclient::sauvegarder ( ) [private]
```

Sauvegarde le contenu du vecteur `clients` dans le fichier `database.txt`.

Cette méthode privée réécrit intégralement le fichier à partir du vecteur `clients`. Le fichier est ouvert en mode écrasement — son contenu précédent est donc effacé à chaque appel. Chaque client est écrit sur deux lignes consécutives : le numéro de carte, puis le nom.

Elle est systématiquement appelée après toute opération modifiant le vecteur (ajout, suppression, modification) afin de garantir la cohérence entre la mémoire et le fichier de persistance.

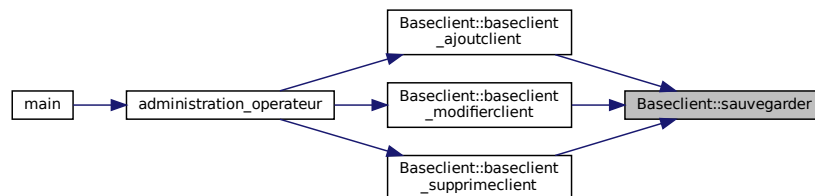
Avertissement

En cas d'échec d'ouverture du fichier, la modification reste présente en mémoire dans le vecteur mais n'est pas persistée sur disque.

Variables internes utilisées :

- **fich** : flux d'écriture associé au fichier `database.txt`.
- **c** : référence constante sur chaque [Client](#) du vecteur lors du parcours.

Voici le graphe des appelants de cette fonction :



6.1.4 Documentation des données membres

6.1.4.1 clients

```
vector<Client> Baseclient::clients [private]
```

6.1.4.2 fichier

```
const string Baseclient::fichier = "database.txt" [private]
```

La documentation de cette classe a été générée à partir du fichier suivant :

- [baseclient.h](#)
- [baseclient.cpp](#)

6.2 Référence de la classe Bouton

Gère la lecture et la remise à zéro des boutons physiques de la borne.

```
#include <bouton.h>
```

Fonctions membres publiques

- `Bouton ()`
Constructeur de la classe Bouton.
- `int bouton_appuie_bouton_charge ()`
Lit l'état du bouton CHARGE.
- `int bouton_appuie_bouton_stop ()`
Lit l'état du bouton STOP.
- `void bouton_set_bouton_charge ()`
Remet à zéro logiciellement le bouton CHARGE.
- `void bouton_set_bouton_stop ()`
Remet à zéro logiciellement le bouton STOP.

Attributs privés

- `entrees * io_b`
Pointeur vers la structure des entrées matérielles de la borne.

6.2.1 Description détaillée

Gère la lecture et la remise à zéro des boutons physiques de la borne.

La classe `Bouton` encapsule les accès aux boutons matériels de la borne de recharge. Elle expose des méthodes de lecture d'état et de remise à zéro logicielle pour chacun des deux boutons disponibles.

En interne, elle utilise un pointeur `io_b` vers la structure `entrees` qui représente le registre d'entrées matérielles de la borne, accessible via un mapping mémoire défini dans `memoire_borne.h`.

6.2.2 Documentation des constructeurs et destructeur

6.2.2.1 Bouton()

```
Bouton::Bouton ( )
```

Constructeur de la classe `Bouton`.

Initialise le pointeur `io_b` vers l'adresse mémoire correspondant aux entrées matérielles de la borne, permettant ainsi l'accès aux états physiques des boutons.

Initialise le pointeur `io_b` en récupérant l'adresse du segment de mémoire partagée via `acces_memoire()`. Ce segment contient la structure `entrees` qui regroupe les états de tous les boutons physiques de la borne.

Une fois le constructeur exécuté, toutes les lectures et écritures sur les boutons transitent directement par ce pointeur, sans nouvel accès au mécanisme de mémoire partagée.

Variables internes utilisées :

- `shmid_b` : identifiant du segment de mémoire partagée, passé par adresse à `acces_memoire()` pour être initialisé.

6.2.3 Documentation des fonctions membres

6.2.3.1 bouton_appuie_button_stop()

```
int Bouton::bouton_appuie_button_stop ( )
```

Lit l'état du bouton STOP.

Lit l'état du bouton STOP depuis le registre matériel.

Interroge le registre d'entrées via `io_b` pour déterminer si le bouton STOP a été actionné. Ce bouton peut être utilisé depuis les états 2, 3 et 4 pour interrompre la charge en cours et rediriger le système vers `etat5` (récupération du véhicule).

Renvoie

int

Valeurs retournées

1	si le bouton STOP est appuyé.
0	si le bouton STOP n'est pas appuyé.

Retourne directement la valeur du champ `bouton_stop` de la structure `entrees` pointée par `io_b`. Aucun traitement n'est effectué — la valeur reflète l'état physique instantané du bouton.

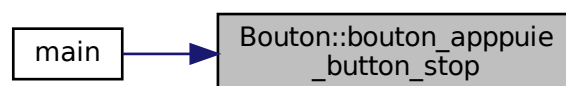
Renvoie

int

Valeurs retournées

1	si le bouton STOP est actuellement appuyé.
0	si le bouton STOP est au repos.

Voici le graphe des appelants de cette fonction :



6.2.3.2 bouton_appuie_button_charge()

```
int Bouton::bouton_appuie_button_charge ( )
```

Lit l'état du bouton CHARGE.

Lit l'état du bouton CHARGE depuis le registre matériel.

Interroge le registre d'entrées via `io_b` pour déterminer si le bouton CHARGE a été appuyé par le client. Ce bouton est surveillé dans `etat0`, où le client dispose de 60 secondes pour l'activer afin de démarrer le cycle de charge.

Renvoie

int

Valeurs retournées

1	si le bouton CHARGE est appuyé.
0	si le bouton CHARGE n'est pas appuyé.

Retourne directement la valeur du champ `bouton_charge` de la structure `entrees` pointée par `io_b`. Aucun traitement n'est effectué — la valeur reflète l'état physique instantané du bouton.

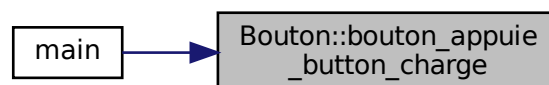
Renvoie

int

Valeurs retournées

1	si le bouton CHARGE est actuellement appuyé.
0	si le bouton CHARGE est au repos.

Voici le graphe des appelants de cette fonction :



6.2.3.3 bouton_set_bouton_charge()

```
void Bouton::bouton_set_bouton_charge ( )
```

Remet à zéro logiciellement le bouton CHARGE.

Réinitialise l'état du bouton CHARGE dans le registre d'entrées après sa prise en compte par le système. Appelée en début de `etat1` pour éviter une détection parasite lors du cycle suivant.

Renvoie

void

Force la valeur du champ `bouton_charge` à 0 dans le registre matériel via le pointeur `io_b`. Cette remise à zéro est nécessaire après la prise en compte de l'appui, pour éviter qu'une lecture ultérieure ne détecte à nouveau un appui fantôme sur le bouton.

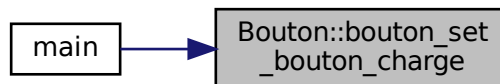
Note

Appelée en début de `etat1`, juste après la confirmation de l'appui bouton détectée dans `etat0`.

Renvoie

void

Voici le graphe des appelants de cette fonction :

**6.2.3.4 bouton_set_bouton_stop()**

```
void Bouton::bouton_set_bouton_stop ( )
```

Remet à zéro logiciellement le bouton STOP.

Réinitialise l'état du bouton STOP dans le registre d'entrées après sa prise en compte par le système. Appelée après chaque détection d'appui sur STOP dans les états 2, 3 et 4, ainsi qu'en initialisation dans le `main` avant le démarrage de la boucle principale.

Renvoie

void

Force la valeur du champ `bouton_stop` à 0 dans le registre matériel via le pointeur `io_b`. Comme pour le bouton CHARGE, cette remise à zéro est indispensable après chaque détection d'appui pour garantir qu'un seul événement est traité par appui physique.

Note

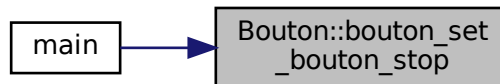
Appelée à deux moments distincts dans le système :

- en initialisation dans le `main`, avant le démarrage de la boucle principale, pour partir d'un état propre.
- après chaque détection d'appui STOP dans les états 2, 3 et 4, avant la transition vers `etat5`.

Renvoie

void

Voici le graphe des appelants de cette fonction :



6.2.4 Documentation des données membres

6.2.4.1 io_b

```
entrees* Bouton::io_b [private]
```

Pointeur vers la structure des entrées matérielles de la borne.

Ce pointeur est initialisé dans le constructeur via le mapping mémoire défini dans `memoire_borne.h`. Il donne accès en lecture et en écriture aux registres d'entrées physiques, notamment les états des boutons CHARGE et STOP.

La documentation de cette classe a été générée à partir du fichier suivant :

- [bouton.h](#)
- [bouton.cpp](#)

6.3 Référence de la structure Client

Représente un client enregistré dans la base de données.

```
#include <baseclient.h>
```

Attributs publics

- int [numero](#)
- string [nom](#)

6.3.1 Description détaillée

Représente un client enregistré dans la base de données.

Chaque client est identifié par un numéro de carte et associé à un nom. Ces deux champs sont stockés sur deux lignes consécutives dans le fichier `database.txt`.

6.3.2 Documentation des données membres

6.3.2.1 nom

`Client::nom`

Nom du client

6.3.2.2 numero

`Client::numero`

Numéro de carte du client. Doit être unique et différent de celui réservé à l'accès opérateur.

La documentation de cette structure a été générée à partir du fichier suivant :

— [baseclient.h](#)

6.4 Référence de la classe `Generateur_Save`

Pilote le générateur PWM, le contacteur AC et la lecture de tension DC de la borne de recharge.

```
#include <generateur_save.h>
```

Fonctions membres publiques

- [Generateur_Save](#) ()
Constructeur de la classe [Generateur_Save](#).
- void [generateur_save_fermer_contacteur](#) ()
Ferme le contacteur AC pour alimenter le véhicule.
- void [generateur_save_ouvrir_contacteur](#) ()
Ouvre le contacteur AC pour couper l'alimentation du véhicule.
- int [generateur_save_tension_DC](#) ()
Lit la tension DC mesurée côté véhicule.
- void [generateur_save_generer_pwm](#) (pwm signal)
Génère un signal PWM selon la phase de charge demandée.

Attributs privés

- entrees * [io_g](#)
Pointeur vers la structure des entrées matérielles de la borne.

6.4.1 Description détaillée

Pilote le générateur PWM, le contacteur AC et la lecture de tension DC de la borne de recharge.

La classe [Generateur_Save](#) encapsule l'ensemble des interactions avec les composants de puissance de la borne. Elle est utilisée dans la machine à états de [borne.cpp](#) pour piloter les transitions entre les différentes phases du cycle de charge :

Phase	Signal PWM	Contacteur	État associé
Attente	STOP	Ouvert	etat0, etat6
Détection	DC	Ouvert	etat1, etat4
Pré-charge	AC_1K	Ouvert	etat2
Charge	AC_CL	Fermé	etat3
Récupération	DC	Ouvert	etat5

En interne, elle s'appuie sur un pointeur `io_g` vers la structure `entrees` mappée en mémoire partagée, donnant accès direct aux registres matériels de la borne.

6.4.2 Documentation des constructeurs et destructeur

6.4.2.1 `Generateur_Save()`

```
Generateur_Save::Generateur_Save ( )
```

Constructeur de la classe [Generateur_Save](#).

Initialise le pointeur `io_g` vers le segment de mémoire partagée contenant les registres matériels de la borne, via `acces_memoire()`.

Récupère l'adresse du segment de mémoire partagée via `acces_memoire()` et l'affecte au pointeur `io_g`. À partir de ce moment, toutes les interactions avec le matériel transitent par ce pointeur sans nouvel accès au mécanisme de mémoire partagée.

Variables internes utilisées :

- **shmid_g** : identifiant du segment de mémoire partagée, passé par adresse à `acces_memoire()` pour être initialisé.

6.4.3 Documentation des fonctions membres

6.4.3.1 `generateur_save_fermer_contacteur()`

```
void Generateur_Save::generateur_save_fermer_contacteur ( )
```

Ferme le contacteur AC pour alimenter le véhicule.

Active le contacteur électromécanique afin d'établir la connexion entre le réseau AC et le chargeur embarqué du véhicule. Cette action est effectuée en `etat3`, une fois le signal PWM `AC_CL` généré et la prise correctement verrouillée.

Avertissement

Ne pas fermer le contacteur sans avoir préalablement généré le signal PWM approprié, au risque d'endommager le chargeur embarqué du véhicule.

Renvoie

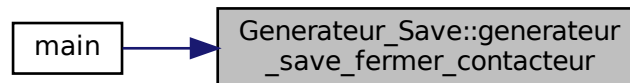
void

Force le champ `contacteur_AC` à 1 dans le registre matériel via `io_g`, ce qui provoque la fermeture physique du contacteur électromécanique et établit la connexion AC vers le véhicule.

Note

Appelée exclusivement en `etat3`, après génération du signal `AC_CL` et verrouillage de la trappe.

Voici le graphe des appelants de cette fonction :

**6.4.3.2 generateur_save_generer_pwm()**

```
void Generateur_Save::generateur_save_generer_pwm (
    pwm signal )
```

Génère un signal PWM selon la phase de charge demandée.

Configure et active le générateur de signal PWM en fonction du paramètre `signal`. Chaque valeur correspond à une phase précise du cycle de charge :

- DC : signal de détection, utilisé en `etat1`, `etat4` et `etat5`.
- AC_1K : signal de pré-charge à 1kHz, utilisé en `etat2`.
- AC_CL : signal de charge active, utilisé en `etat3`.
- STOP : arrêt du générateur, utilisé en `etat6` et lors d'un appui sur le bouton STOP.

Paramètres

<i>signal</i>	Valeur de type <code>pwm</code> indiquant la phase à générer.
---------------	---

Renvoie

void

Affecte directement la valeur de `signal` au champ `gene_pwm` du registre matériel via `io_g`. Le matériel prend en charge la génération effective du signal PWM correspondant sans traitement supplémentaire côté logiciel.

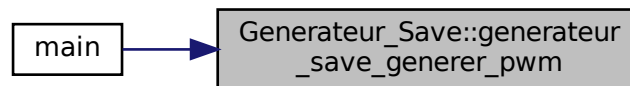
Correspondance entre les valeurs du type `pwm` et leur usage :

- `DC` : détection de présence véhicule — `etat1`, `etat4`, `etat5`.
- `AC_1K` : pré-charge à 1kHz — `etat2`.
- `AC_CL` : charge active — `etat3`.
- `STOP` : arrêt du générateur — `etat6` et appui bouton `STOP`.

Avertissement

Changer le signal PWM sans ouvrir le contacteur au préalable peut provoquer des transitoires de tension dangereuses pour le chargeur embarqué du véhicule.

Voici le graphe des appelants de cette fonction :



6.4.3.3 `generateur_save_ouvrir_contacteur()`

```
void Generateur_Save::generateur_save_ouvrir_contacteur ( )
```

Ouvre le contacteur AC pour couper l'alimentation du véhicule.

Désactive le contacteur électromécanique afin d'interrompre la connexion entre le réseau AC et le véhicule. Appelée lors de toute transition nécessitant une coupure sécurisée du circuit, notamment en `etat2`, `etat4` et `etat6`.

Renvoie

`void`

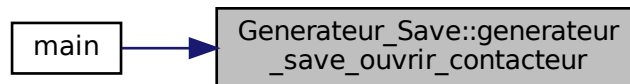
Force le champ `contacteur_AC` à 0 dans le registre matériel via `io_g`, ce qui provoque l'ouverture physique du contacteur électromécanique et coupe la connexion AC vers le véhicule.

Note

Appelée dans plusieurs états du cycle :

- en `etat2` : ouverture avant le passage en charge active.
- en `etat4` : ouverture en fin de charge lors de la récupération.
- en `etat6` : ouverture finale avant remise en disponibilité de la borne.

Voici le graphe des appelants de cette fonction :



6.4.3.4 generateur_save_tension_DC()

```
int Generateur_Save::generateur_save_tension_DC ( )
```

Lit la tension DC mesurée côté véhicule.

Retourne la valeur entière de la tension DC lue depuis le registre matériel via `io_g`. Cette mesure est utilisée à chaque état du cycle de charge pour décider des transitions :

- retourne **9** → prise détectée, transition vers `etat2`.
- retourne **6** → véhicule prêt à charger, transition vers `etat3`.
- retourne **9** → charge terminée, transition vers `etat4`.
- retourne **12** → véhicule débranché, transition vers `etat6`.

Renvoie

int Valeur entière de la tension DC mesurée.

Retourne directement la valeur du champ `gene_u` depuis le registre matériel via `io_g`. Ce champ représente la tension DC mesurée côté véhicule, mise à jour en temps réel par le matériel.

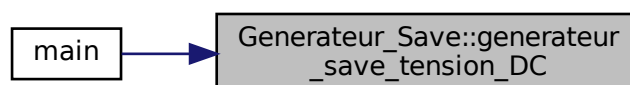
Les valeurs attendues et leurs significations dans le cycle de charge :

- **9** → prise détectée par le véhicule → transition `etat1` vers `etat2`.
- **6** → véhicule prêt à recevoir la charge → transition `etat2` vers `etat3`.
- **9** → charge terminée → transition `etat3` vers `etat4`.

Note — **12** → véhicule débranché → transition `etat5` vers `etat6`.

Toute valeur différente de celles listées ci-dessus maintient le système dans son état courant jusqu'à la prochaine lecture.

Voici le graphe des appelants de cette fonction :



6.4.4 Documentation des données membres

6.4.4.1 io_g

```
entrees* Generateur_Save::io_g [private]
```

Pointeur vers la structure des entrées matérielles de la borne.

Initialisé dans le constructeur via `acces_memoire()`, ce pointeur donne accès en lecture et en écriture aux registres matériels, notamment la tension DC mesurée et les commandes PWM et contacteur.

La documentation de cette classe a été générée à partir du fichier suivant :

- [generateur_save.h](#)
- [generateur_save.cpp](#)

6.5 Référence de la classe LecteurCarte

Gère l'initialisation et la lecture du lecteur de cartes.

```
#include <lecteurcarte.h>
```

Fonctions membres publiques

- void [lecteurcarte_initialiser_lecteur\(\)](#)
Initialise le port de communication du lecteur de cartes.
- void [lecteurcarte_initialiser\(\)](#)
Initialise une nouvelle session de lecture de carte.
- int [lecteurcarte_lire_carte\(\)](#)
Lit le numéro de la carte présentée au lecteur.

6.5.1 Description détaillée

Gère l'initialisation et la lecture du lecteur de cartes.

La classe [LecteurCarte](#) encapsule les interactions avec le lecteur de cartes de la borne. Elle est utilisée exclusivement dans `etat0` de la machine à états, où elle constitue le point d'entrée obligatoire de tout cycle de charge : sans carte valide présentée et reconnue, aucune action n'est possible sur la borne.

Le cycle d'utilisation typique est le suivant :

- 1. [lecteurcarte_initialiser_lecteur\(\)](#) — appelée une fois dans le `main` avant la boucle principale pour ouvrir le port.
- 2. [lecteurcarte_initialiser\(\)](#) — appelée à chaque entrée en `etat0` pour préparer une nouvelle session de lecture.
- 3. [lecteurcarte_lire_carte\(\)](#) — appelée après initialisation pour lire le numéro de la carte présentée.

Avertissement

Les méthodes [lecteurcarte_initialiser\(\)](#) et [lecteurcarte_lire_carte\(\)](#) ne doivent pas être appelées avant [lecteurcarte_initialiser_lecteur\(\)](#), sous peine d'un accès à un port non ouvert.

6.5.2 Documentation des fonctions membres

6.5.2.1 lecteurcarte_initialiser()

```
void LecteurCarte::lecteurcarte_initialiser ( )
```

Initialise une nouvelle session de lecture de carte.

Prépare le lecteur pour une nouvelle lecture en réinitialisant son état interne. Doit être appelée à chaque entrée en `etat0`, c'est-à-dire au début de chaque nouveau cycle client, y compris après un retour depuis `etat6` ou `etat255`.

Renvoie

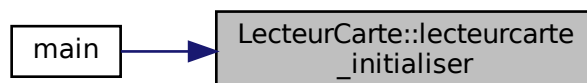
void

Dans cette version simulée, l'initialisation se limite à afficher un message invitant l'utilisateur à saisir son numéro de carte au clavier. Sur le matériel réel, cette méthode sera chargée de réinitialiser l'état interne du lecteur avant chaque nouvelle session.

Note

Appelée à chaque entrée en `etat0`, y compris après un retour depuis `etat6` ou `etat255`.

Voici le graphe des appelants de cette fonction :



6.5.2.2 lecteurcarte_initialiser_lecteur()

```
void LecteurCarte::lecteurcarte_initialiser_lecteur ( )
```

Initialise le port de communication du lecteur de cartes.

Ouvre et configure le port de communication utilisé par le lecteur de cartes, via les primitives fournies par `lcarte.h`. Cette initialisation est globale et ne doit être effectuée qu'une seule fois, au démarrage du système, avant le lancement de la boucle principale dans le `main`.

Avertissement

Appeler cette méthode plusieurs fois en cours d'exécution peut provoquer des conflits d'accès au port de communication.

Renvoie

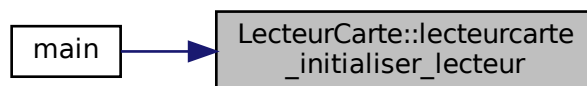
void

Dans cette version du système, le corps de la méthode est volontairement laissé vide. L'initialisation du port de communication sera implémentée lors de l'intégration du lecteur de cartes physique, via les primitives de `lcarte.h`.

Note

Cette méthode est prévue pour accueillir la configuration du port série ou du bus de communication du lecteur lors du passage en environnement matériel réel.

Voici le graphe des appelants de cette fonction :

**6.5.2.3 lecteurcarte_lire_carte()**

```
int LecteurCarte::lecteurcarte_lire_carte ( )
```

Lit le numéro de la carte présentée au lecteur.

Attend qu'une carte soit présentée et retourne son numéro identifiant. Ce numéro est ensuite transmis à `baseclient_authentifier()` pour vérification dans la base clients.

Deux cas particuliers sont à noter :

- **255** : carte opérateur détectée — déclenche la transition vers `etat255` pour accéder au mode administration.
- **tout autre numéro** : numéro client transmis à l'authentification.

Renvoie

int Numéro lu depuis la carte présentée.

Valeurs retournées

255	Carte opérateur détectée.
-----	---------------------------

Dans cette version simulée, la lecture est effectuée par saisie clavier via `cin`. Le numéro saisi est retourné direc-

tement sans traitement, de la même façon que le ferait un lecteur physique.

Sur le matériel réel, cette méthode sera remplacée par un appel aux primitives de `lcarte.h` pour récupérer le numéro depuis le lecteur.

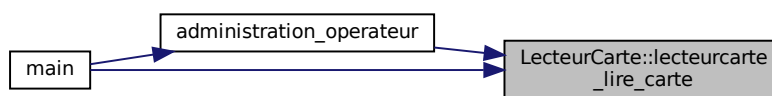
Variables internes utilisées :

- **num** : entier int stockant temporairement le numéro saisi avant retour à l'appelant.

Note

La valeur 255 saisie ici déclenche la transition vers `etat255` dans la machine à états — elle est réservée à l'opérateur et ne doit pas être attribuée à un client dans la base de données.

Voici le graphe des appelants de cette fonction :



La documentation de cette classe a été générée à partir du fichier suivant :

- [lecteurcarte.h](#)
- [lecteurcarte.cpp](#)

6.6 Référence de la classe Prise

Pilote la trappe mécanique et le voyant de la prise de charge.

```
#include <prise.h>
```

Fonctions membres publiques

- [Prise](#) ()
Constructeur de la classe [Prise](#).
- void [prise_verrouille_trappe](#) ()
Verrouille mécaniquement la trappe de la prise de charge.
- void [prise_deverrouille_trappe](#) ()
Déverrouille mécaniquement la trappe de la prise de charge.
- void [prise_set_prise](#) (led prise)
Pilote le voyant lumineux de la prise de charge.

Attributs privés

- entrees * [io_p](#)
Pointeur vers la structure des entrées matérielles de la borne.

6.6.1 Description détaillée

Pilote la trappe mécanique et le voyant de la prise de charge.

La classe [Prise](#) encapsule le contrôle des actionneurs physiques de la prise de charge de la borne. Elle intervient à plusieurs étapes du cycle de charge pour garantir la sécurité de la connexion :

Action	Méthode	État associé
Déverrouillage trappe	<code>prise_deverrouille_trappe()</code>	etat1, etat4, etat5
Voyant prise allumé	<code>prise_set_prise(VERT)</code>	etat1
Verrouillage trappe	<code>prise_verrouille_trappe()</code>	etat2, etat6
Voyant prise éteint	<code>prise_set_prise(OFF)</code>	etat5

En interne, elle s'appuie sur un pointeur `io_p` vers la structure `entrees` mappée en mémoire partagée, donnant accès direct aux registres matériels de la borne.

Avertissement

Le verrouillage de la trappe ne doit jamais être effectué avant que le câble soit correctement branché au véhicule, sous peine de bloquer mécaniquement la prise dans un état incohérent.

6.6.2 Documentation des constructeurs et destructeur

6.6.2.1 Prise()

```
Prise::Prise ( )
```

Constructeur de la classe `Prise`.

Initialise le pointeur `io_p` vers le segment de mémoire partagée contenant les registres matériels de la borne, via `acces_memoire()`.

Récupère l'adresse du segment de mémoire partagée via `acces_memoire()` et l'affecte au pointeur `io_p`. À partir de ce moment, toutes les interactions avec les actionneurs physiques de la prise transitent par ce pointeur sans nouvel accès au mécanisme de mémoire partagée.

Variables internes utilisées :

- **shmid_p** : identifiant du segment de mémoire partagée, passé par adresse à `acces_memoire()` pour être initialisé.

6.6.3 Documentation des fonctions membres

6.6.3.1 prise_deverrouille_trappe()

```
void Prise::prise_deverrouille_trappe ( )
```

Déverrouille mécaniquement la trappe de la prise de charge.

Désactive le verrou mécanique de la trappe pour permettre au client de brancher ou débrancher le câble de charge. Appelée avant toute opération nécessitant un accès physique à la prise.

Renvoie

void

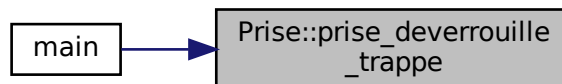
Force le champ `led_trappe` à VERT dans le registre matériel via `io_p`, ce qui provoque l'ouverture physique du verrou de la trappe et signale visuellement au client qu'il peut accéder à la prise.

Note

Appelée dans trois états distincts du cycle :

- en `etat1` : déverrouillage pour permettre le branchement du câble.
- en `etat4` : déverrouillage en fin de charge pour la récupération.
- en `etat5` : déverrouillage pour permettre le débranchement du câble.

Voici le graphe des appelants de cette fonction :



6.6.3.2 prise_set_prise()

```
void Prise::prise_set_prise (
    led prise )
```

Pilote le voyant lumineux de la prise de charge.

Configure l'état du voyant de la prise selon la valeur du paramètre `prise`, de type `led`. Les états possibles sont :

- VERT : prise active, câble correctement détecté — `etat1`.
- OFF : prise inactive, fin de session ou récupération du véhicule — `etat5`.

Paramètres

<i>prise</i>	Valeur de type <code>led</code> indiquant l'état à appliquer sur le voyant de la prise.
--------------	---

Renvoie

void

Affecte directement la valeur de `prise` au champ `led_prise` du registre matériel via `io_p`, ce qui configure l'état visuel du voyant de la prise sans traitement supplémentaire côté logiciel.

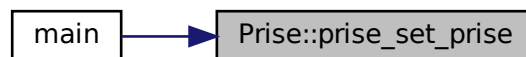
Les valeurs utilisées dans le cycle de charge :

- VERT : prise active, câble correctement détecté — `etat1`.
- OFF : prise inactive, fin de session ou récupération — `etat5`.

Note

Le champ `led_prise` est distinct du champ `led_trappe` — le voyant de la prise et l'actionneur de la trappe sont deux registres matériels indépendants.

Voici le graphe des appelants de cette fonction :

**6.6.3.3 prise_verrouille_trappe()**

```
void Prise::prise_verrouille_trappe ( )
```

Verrouille mécaniquement la trappe de la prise de charge.

Active le verrou mécanique de la trappe pour sécuriser la connexion entre le câble de charge et le véhicule. Cette action empêche tout débranchement accidentel ou volontaire durant la charge.

Avertissement

Ne pas verrouiller la trappe avant de s'être assuré que le câble est correctement inséré dans la prise du véhicule.

Renvoie

void

Force le champ `led_trappe` à OFF dans le registre matériel via `io_p`, ce qui provoque la fermeture physique du verrou de la trappe.

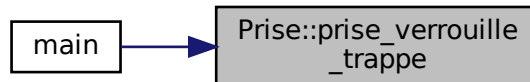
Note

Le choix d'utiliser le champ `led_trappe` pour piloter le verrou mécanique traduit une conception matérielle où l'actionneur de la trappe est commandé via le même registre que son voyant associé.

Appelée dans deux états distincts du cycle :

- en `etat2` : verrouillage une fois le câble branché au véhicule.
- en `etat6` : verrouillage final avant remise en disponibilité de la borne.

Voici le graphe des appelants de cette fonction :



6.6.4 Documentation des données membres

6.6.4.1 io_p

```
entrees* Prise::io_p [private]
```

Pointeur vers la structure des entrées matérielles de la borne.

Initialisé dans le constructeur via `acces_memoire()`, ce pointeur donne accès en lecture et en écriture aux registres matériels, notamment la commande de la trappe et l'état du voyant de la prise.

La documentation de cette classe a été générée à partir du fichier suivant :

- [prise.h](#)
- [prise.cpp](#)

6.7 Référence de la classe Timer

Gère la temporisation et la mesure du temps écoulé dans la borne.

```
#include <timer.h>
```

Fonctions membres publiques

- [Timer](#) ()
Constructeur de la classe [Timer](#).
- void [timer_raz](#) ()
Remet à zéro le timer logiciel.
- int [timer_valeur](#) ()
Lit la valeur courante du timer matériel.
- int [timer_count_sec](#) ()
Calcule le nombre de secondes écoulées depuis la dernière remise à zéro.

Attributs privés

- entrees * `io_t`
Pointeur vers la structure des entrées matérielles de la borne.
- `time_t` `depart_timer`
Horodatage de référence pour la mesure du temps écoulé.

6.7.1 Description détaillée

Gère la temporisation et la mesure du temps écoulé dans la borne.

La classe `Timer` encapsule deux mécanismes de mesure du temps : un accès au timer matériel via le pointeur `io_t`, et un horodatage logiciel via le membre `depart_timer` de type `time_t`.

Son utilisation typique dans la machine à états est la suivante :

- 1. `timer_raz()` — remise à zéro au début de la temporisation.
- 2. `timer_count_sec()` — appelée en boucle pour accumuler les secondes écoulées jusqu'à expiration du délai.
- 3. `timer_valeur()` — lecture directe de la valeur courante du timer matériel si nécessaire.

Note

La temporisation de 60 secondes dans `etat0` est entièrement gérée par cette classe — c'est la seule contrainte temporelle explicite du cycle de charge côté logiciel.

6.7.2 Documentation des constructeurs et destructeur

6.7.2.1 Timer()

```
Timer::Timer ( )
```

Constructeur de la classe `Timer`.

Initialise le pointeur `io_t` vers le segment de mémoire partagée contenant les registres matériels de la borne, via `acces_memoire()`.

Récupère l'adresse du segment de mémoire partagée via `acces_memoire()` et l'affecte au pointeur `io_t`. À partir de ce moment, toutes les interactions avec le registre timer matériel transitent par ce pointeur sans nouvel accès au mécanisme de mémoire partagée.

Variables internes utilisées :

- `shmid_t` : identifiant du segment de mémoire partagée, passé par adresse à `acces_memoire()` pour être initialisé.

6.7.3 Documentation des fonctions membres

6.7.3.1 timer_count_sec()

```
int Timer::timer_count_sec ( )
```

Calcule le nombre de secondes écoulées depuis la dernière remise à zéro.

Calcule et retourne la différence en secondes entre l'instant courant et `depart_timer`. Cette valeur est accumulée dans la variable `timer_secs` de `borne.cpp` à chaque tour de la boucle d'attente, jusqu'à atteindre le seuil de 60 secondes ou détecter un appui sur le bouton CHARGE.

Renvoie

int Nombre de secondes écoulées depuis le dernier appel à `timer_raz()`.

Appelle `timer_valeur()` pour obtenir le nombre de secondes écoulées depuis le dernier appel, puis cumule cette valeur dans le registre matériel `timer_sec` via `io_t`, assurant ainsi la cohérence entre le compteur logiciel et le compteur hardware.

C'est cette méthode qui est appelée à chaque tour de la boucle d'attente dans `etat0` — sa valeur de retour est accumulée dans `timer_secs` jusqu'à atteindre 60 secondes ou détecter un appui sur le bouton CHARGE.

Variables internes utilisées :

- **timer_value** : entier stockant temporairement la valeur retournée par `timer_valeur()` avant cumul et retour.

Renvoie

int Nombre de secondes écoulées depuis le dernier appel à `timer_valeur()`.

Voici le graphe d'appel pour cette fonction :



Voici le graphe des appelants de cette fonction :



6.7.3.2 timer_raz()

```
void Timer::timer_raz ( )
```

Remet à zéro le timer logiciel.

Enregistre l'instant courant dans `depart_timer` via `time()`, constituant ainsi le point de départ de la temporisation. Doit être appelée au début de chaque période de temporisation, notamment en `etat0` avant la boucle d'attente d'appui bouton de 60 secondes.

Renvoie

void

Effectue une double remise à zéro :

- le registre matériel `timer_sec` est forcé à 0 via `io_t`, ce qui réinitialise le compteur côté hardware.
- `depart_timer` est mis à jour avec l'instant courant via `time(NULL)`, ce qui constitue le nouveau point de départ pour les calculs d'intervalles effectués par `timer_valeur()`.

Note

Appelée en `etat0` juste avant la boucle d'attente d'appui bouton, pour garantir que la temporisation de 60 secondes part bien de zéro à chaque nouveau cycle client.

Voici le graphe des appelants de cette fonction :



6.7.3.3 timer_valeur()

```
int Timer::timer_valeur ( )
```

Lit la valeur courante du timer matériel.

Retourne directement la valeur du registre timer hardware de la borne via le pointeur `io_t`, sans calcul supplémentaire.

Renvoie

int Valeur courante du timer matériel.

Attend une seconde via `sleep(1)`, puis calcule la différence entre l'instant courant et `depart_timer`. Cette différence représente le temps réellement écoulé depuis le dernier appel à cette méthode. `depart_timer` est ensuite mis à jour avec l'instant courant pour préparer le prochain appel.

Note

L'appel à `sleep(1)` est intentionnel — il cadence la boucle d'attente dans `etat0` à une itération par seconde, évitant ainsi une occupation CPU inutile durant la temporisation.

Avertissement

La valeur retournée peut légèrement dépasser 1 seconde en cas de charge système importante, `sleep()` n'étant pas une fonction temps réel garantie à la milliseconde près.

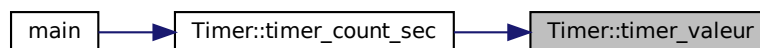
Variables internes utilisées :

- **now** : horodatage de l'instant courant obtenu via `time(NULL)`.
- **diff** : différence en secondes entre `now` et `depart_timer`.

Renvoie

int Nombre de secondes écoulées depuis le dernier appel.

Voici le graphe des appelants de cette fonction :



6.7.4 Documentation des données membres

6.7.4.1 depart_timer

```
time_t Timer::depart_timer [private]
```

Horodatage de référence pour la mesure du temps écoulé.

Stocke l'instant de la dernière remise à zéro du timer, obtenu via `time()`. La différence entre l'heure courante et cette valeur de référence donne le temps écoulé en secondes.

6.7.4.2 io_t

```
entrees* Timer::io_t [private]
```

Pointeur vers la structure des entrées matérielles de la borne.

Initialisé dans le constructeur via `acces_memoire()`, ce pointeur donne accès en lecture aux registres matériels, notamment la valeur courante du timer hardware de la borne.

La documentation de cette classe a été générée à partir du fichier suivant :

- [timer.h](#)
- [timer.cpp](#)

6.8 Référence de la classe Voyant

Pilote l'ensemble des voyants lumineux de la borne de recharge.

```
#include <voyant.h>
```

Fonctions membres publiques

- [Voyant](#) ()
Constructeur de la classe [Voyant](#).
- void [voyant_set_dispo](#) (led dispo)
Pilote le voyant de disponibilité de la borne.
- void [voyant_set_charge](#) (led charge)
Pilote le voyant de charge de la borne.
- void [voyant_blink_charge](#) ()
Fait clignoter le voyant de charge pendant 8 secondes.
- void [voyant_set_default](#) (led default)
Pilote le voyant de défaut de la borne.
- void [voyant_blink_default](#) ()
Fait clignoter le voyant de défaut pendant 8 secondes.
- void [voyant_set_prise](#) (led prise)
Pilote le voyant lumineux de la prise de charge.
- void [voyant_set_trappe](#) (led trappe)
Pilote le voyant lumineux de la trappe de la prise.

Attributs privés

- entrees * [io](#)
Pointeur vers la structure des entrées matérielles de la borne.

6.8.1 Description détaillée

Pilote l'ensemble des voyants lumineux de la borne de recharge.

La classe [Voyant](#) encapsule le contrôle de tous les indicateurs lumineux de la borne. Elle intervient à chaque transition de la machine à états pour informer visuellement le client de l'évolution du cycle de charge :

Voyant	Méthode	État associé
Disponibilité	voyant_set_dispo (VERT)	etat6
Disponibilité	voyant_set_dispo (OFF)	etat1
Charge	voyant_set_charge (ROUGE)	etat1
Charge	voyant_set_charge (VERT)	etat4
Charge	voyant_set_charge (OFF)	etat5
Charge	voyant_blink_charge ()	etat0 (succès auth)
Défaut	voyant_blink_default ()	etat0 (échec auth)

En interne, elle s'appuie sur un pointeur [io](#) vers la structure [entrees](#) mappée en mémoire partagée, donnant accès direct aux registres matériels des voyants.

6.8.2 Documentation des constructeurs et destructeur

6.8.2.1 Voyant()

```
Voyant::Voyant ( )
```

Constructeur de la classe [Voyant](#).

Initialise le pointeur `io` vers le segment de mémoire partagée contenant les registres matériels de la borne, via `acces_memoire()`.

Récupère l'adresse du segment de mémoire partagée via `acces_memoire()` et l'affecte au pointeur `io`. À partir de ce moment, toutes les interactions avec les registres matériels des voyants transitent par ce pointeur sans nouvel accès au mécanisme de mémoire partagée.

Variables internes utilisées :

- **shmid** : identifiant du segment de mémoire partagée, passé par adresse à `acces_memoire()` pour être initialisé.

6.8.3 Documentation des fonctions membres

6.8.3.1 voyant_blink_charge()

```
void Voyant::voyant_blink_charge ( )
```

Fait clignoter le voyant de charge pendant 8 secondes.

Produit un clignotement du voyant de charge pour signaler au client qu'il a été authentifié avec succès et qu'il dispose de 60 secondes pour appuyer sur le bouton CHARGE. Appelée en `etat0` après une authentification réussie.

Avertissement

Cette méthode est bloquante — elle monopolise le fil d'exécution pendant toute la durée du clignotement (8 secondes).

Renvoie

void

Produit 4 cycles d'allumage/extinction du voyant de charge, chaque demi-période durant 1 seconde via `sleep(1)`, pour un clignotement total de 8 secondes. Le voyant alterne entre `VERT` et `OFF` à chaque itération via des appels à `voyant_set_charge()`.

Ce clignotement signale au client qu'il a été authentifié avec succès et qu'il dispose de 60 secondes pour appuyer sur le bouton CHARGE.

Note

Appelée en `etat0` immédiatement après une authentification réussie, avant le démarrage de la temporisation de 60 secondes.

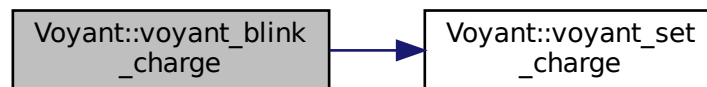
Avertissement

Cette méthode est bloquante pendant 8 secondes — aucune instruction du programme n'est exécutée durant son déroulement. La temporisation de 60 secondes ne démarre qu'après son retour.

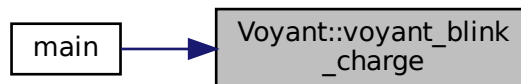
Variables internes utilisées :

— `i` : compteur de boucle de type `int`, itère de 0 à 3 inclus pour produire les 4 cycles de clignotement.

Voici le graphe d'appel pour cette fonction :



Voici le graphe des appelants de cette fonction :

**6.8.3.2 voyant_blink_default()**

```
void Voyant::voyant_blink_default ( )
```

Fait clignoter le voyant de défaut pendant 8 secondes.

Produit un clignotement du voyant de défaut pour signaler au client que son authentification a échoué — carte inconnue ou numéro non reconnu dans la base clients. Appelée en `etat0` après un échec d'authentification.

Avertissement

Cette méthode est bloquante — elle monopolise le fil d'exécution pendant toute la durée du clignotement (8 secondes).

Renvoie

void

Produit 4 cycles d'allumage/extinction du voyant de défaut, chaque demi-période durant 1 seconde via `sleep(1)`, pour un clignotement total de 8 secondes. Le voyant alterne entre ROUGE et OFF à chaque itération via des appels à `voyant_set_default()`.

Ce clignotement signale au client que son authentification a échoué — carte inconnue ou numéro absent de la base clients — avant de retourner en `etat0` pour une nouvelle tentative.

Note

Appelée en `etat0` immédiatement après un échec d'authentification, avant le retour automatique en début de `etat0`.

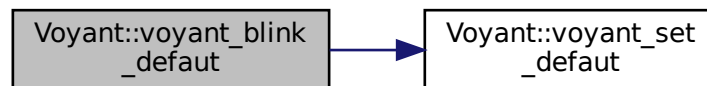
Avertissement

Cette méthode est bloquante pendant 8 secondes — aucune instruction du programme n'est exécutée durant son déroulement.

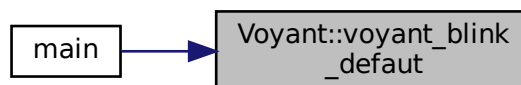
Variables internes utilisées :

— `i` : compteur de boucle de type `int`, itère de 0 à 3 inclus pour produire les 4 cycles de clignotement.

Voici le graphe d'appel pour cette fonction :



Voici le graphe des appelants de cette fonction :

**6.8.3.3 voyant_set_charge()**

```
void Voyant::voyant_set_charge (
    led charge )
```

Pilote le voyant de charge de la borne.

Configure l'état du voyant indiquant la progression du cycle de charge. Les états utilisés dans le cycle sont :

- ROUGE : charge en cours — `etat1`.
- VERT : charge terminée — `etat4`.
- OFF : cycle terminé — `etat5`.

Paramètres

<i>charge</i>	Valeur de type <code>led</code> à appliquer sur le voyant de charge (ROUGE, VERT ou OFF).
---------------	---

Renvoie

`void`

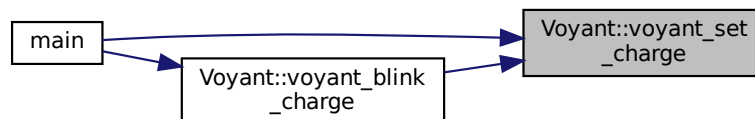
Affecte directement la valeur de `charge` au champ `led_charge` du registre matériel via `io`, sans traitement supplémentaire côté logiciel.

Note

Appelée dans trois états distincts du cycle :

- en `etat1` : allumage rouge (ROUGE) au démarrage de la charge.
- en `etat4` : allumage vert (VERT) en fin de charge.
- en `etat5` : extinction (OFF) lors de la récupération.

Voici le graphe des appelants de cette fonction :



6.8.3.4 voyant_set_default()

```
void Voyant::voyant_set_default (
    led default )
```

Pilote le voyant de défaut de la borne.

Configure l'état du voyant signalant une anomalie ou un échec. Utilisé pour indiquer visuellement un état d'erreur persistant distinct du clignotement temporaire géré par `voyant_blink_default()`.

Paramètres

<i>default</i>	Valeur de type <code>led</code> à appliquer sur le voyant de défaut.
----------------	--

Renvoie

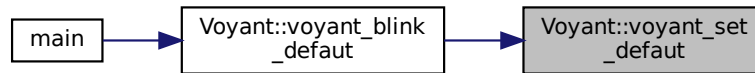
`void`

Affecte directement la valeur de `default` au champ `led_default` du registre matériel via `io`, sans traitement supplémentaire côté logiciel.

Note

Utilisée en interne par `voyant_blink_default()` pour alterner l'état du voyant de défaut durant le clignotement. Peut également être appelée directement pour fixer un état de défaut persistant.

Voici le graphe des appelants de cette fonction :

**6.8.3.5 voyant_set_dispo()**

```
void Voyant::voyant_set_dispo (
    led dispo )
```

Pilote le voyant de disponibilité de la borne.

Configure l'état du voyant indiquant si la borne est disponible pour un nouveau client. Allumé en vert en `etat6` pour signaler que la borne est prête, éteint en `etat1` dès le démarrage d'un cycle de charge.

Paramètres

<i>dispo</i>	Valeur de type <code>led</code> à appliquer sur le voyant de disponibilité (VERT ou OFF).
--------------	---

Renvoie

void

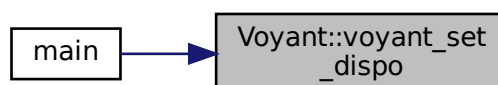
Affecte directement la valeur de `dispo` au champ `led_dispo` du registre matériel via `io`, sans traitement supplémentaire côté logiciel.

Note

Appelée dans deux états distincts du cycle :

- en `etat1` : extinction (OFF) au démarrage du cycle de charge.
- en `etat6` : allumage (VERT) pour signaler la disponibilité de la borne au client suivant.

Voici le graphe des appelants de cette fonction :



6.8.3.6 voyant_set_prise()

```
void Voyant::voyant_set_prise (
    led prise )
```

Pilote le voyant lumineux de la prise de charge.

Configure l'état du voyant associé à la prise de charge, indépendamment du voyant de charge principal. Permet un retour visuel localisé directement au niveau de la prise.

Paramètres

<i>prise</i>	Valeur de type <code>led</code> à appliquer sur le voyant de la prise.
--------------	--

Renvoie

`void`

Affecte directement la valeur de `prise` au champ `led_prise` du registre matériel via `io`, sans traitement supplémentaire côté logiciel.

Note

Ce champ `led_prise` est partagé avec la classe [Prise](#) via le même segment de mémoire partagée — toute écriture ici est donc immédiatement visible depuis `prise_set_prise()` et inversement.

6.8.3.7 voyant_set_trappe()

```
void Voyant::voyant_set_trappe (
    led trappe )
```

Pilote le voyant lumineux de la trappe de la prise.

Configure l'état du voyant associé à la trappe de la prise, fournissant un retour visuel sur l'état du verrou mécanique directement au niveau de la trappe.

Paramètres

<i>trappe</i>	Valeur de type <code>led</code> à appliquer sur le voyant de la trappe.
---------------	---

Renvoie

`void`

Affecte directement la valeur de `trappe` au champ `led_trappe` du registre matériel via `io`, sans traitement supplémentaire côté logiciel.

Note

Ce champ `led_trappe` est partagé avec la classe [Prise](#) via le même segment de mémoire partagée — toute écriture ici est donc immédiatement visible depuis `prise_verrouille_trappe()` et `prise_deverrouille_trappe()` et inversement.

6.8.4 Documentation des données membres

6.8.4.1 io

```
entrees* Voyant::io [private]
```

Pointeur vers la structure des entrées matérielles de la borne.

Initialisé dans le constructeur via `acces_memoire()`, ce pointeur donne accès en écriture aux registres matériels de tous les voyants de la borne, regroupés dans la structure `entrees`.

La documentation de cette classe a été générée à partir du fichier suivant :

- [voyant.h](#)
- [voyant.cpp](#)

Chapitre 7

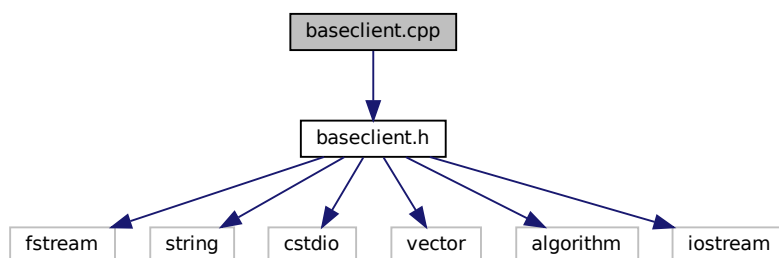
Documentation des fichiers

7.1 Référence du fichier baseclient.cpp

Implémentation des méthodes de la classe [Baseclient](#).

```
#include "baseclient.h"
```

Graphe des dépendances par inclusion de baseclient.cpp:



7.1.1 Description détaillée

Implémentation des méthodes de la classe [Baseclient](#).

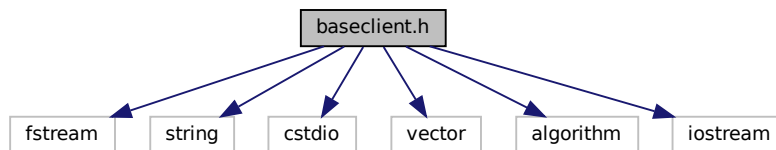
Ce fichier contient l'implémentation complète de la gestion de la base de données clients. Toutes les opérations (lecture, écriture, recherche) transitent par le vecteur `clients` maintenu en mémoire, le fichier `database.txt` n'étant accédé qu'au chargement initial et après chaque modification.

7.2 Référence du fichier baseclient.h

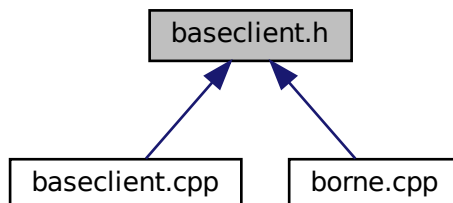
Déclaration de la structure [Client](#) et de la classe [Baseclient](#).

```
#include <fstream>
#include <string>
#include <cstdio>
#include <vector>
#include <algorithm>
#include <iostream>
```

Graphe des dépendances par inclusion de baseclient.h:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



Classes

- struct [Client](#)
Représente un client enregistré dans la base de données.
- class [Baseclient](#)
Classe de gestion de la base de données clients de la borne.

Macros

- #define [IOSTREAM](#)

7.2.1 Description détaillée

Déclaration de la structure [Client](#) et de la classe [Baseclient](#).

Ce fichier définit la structure de données représentant un client ainsi que la classe [Baseclient](#) qui gère l'ensemble des opérations sur la base de données clients (authentification, ajout, suppression, modification et affichage).

La base de données est persistée dans un fichier texte **database.txt**, chargé en mémoire au démarrage dans un vecteur de clients, ce qui évite des accès disque répétés lors des opérations courantes.

7.2.2 Documentation des macros

7.2.2.1 IOSTREAM

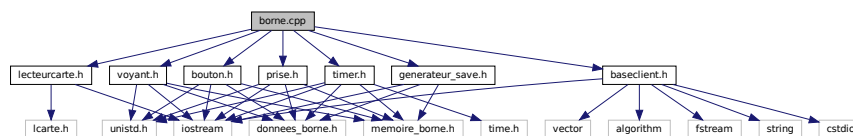
```
#define IOSTREAM
```

7.3 Référence du fichier borne.cpp

Fichier principal du système de borne de recharge Ce fichier contient les objets de chaque classe.

```
#include "lecteurcarte.h"
#include "voyant.h"
#include "baseclient.h"
#include "timer.h"
#include "bouton.h"
#include "prise.h"
#include "generateur_save.h"
```

Graphe des dépendances par inclusion de borne.cpp:



Énumérations

```
— enum etatsystem {
    etat0 , etat1 , etat2 , etat3 ,
    etat4 , etat5 , etat6 , etat255 }
```

Définition des états du système de recharge On déclare une énumération pour représenter les différents états du système de recharge de véhicule électrique.

Fonctions

- void `administration_operateur` ()
Fonction chargée de gérer l'administration des clients une fois que l'opérateur met son id.
- int `main` ()
Fonction principale du système — implémentation de la machine à états décrivant le comportement complet de la borne de recharge.

Variables

- `Baseclient baseclient`
- `LecteurCarte lecteurcarte`

7.3.1 Description détaillée

Fichier principal du système de borne de recharge Ce fichier contient les objets de chaque classe.

Auteurs

- KAMDA TEZEBO DIBREY JONATAN
- Nassime BOUANANI

Version

1.0

Date

30/03/2026

7.3.2 Documentation du type de l'énumération

7.3.2.1 etatsystem

enum `etatsystem`

Définition des états du système de recharge On déclare une énumération pour représenter les différents états du système de recharge de véhicule électrique.

Valeurs énumérées

etat0	
etat1	
etat2	
etat3	
etat4	
etat5	
etat6	
etat255	

7.3.3 Documentation des fonctions

7.3.3.1 administration_operateur()

```
void administration_operateur ( )
```

Fonction chargée de gérer l'administration des clients une fois que l'opérateur met son id.

Note

Cette fonction est utilisée pour le cas etat255.

Cette fonction affiche un menu à l'opérateur avec les opérations disponibles :

- Ajouter un client (choix 1) : demande le numéro de carte et le nom du client, puis appelle baseclient_↵ ajoutclient(int, string).
- Supprimer un client (choix 2) : demande le numéro de carte du client, puis appelle baseclient_↵ supprimeclient(int).
- Afficher tous les clients (choix 3) : appelle baseclient_afficher_clients().
- Modifier un client (choix 4) : demande le numéro de carte du client à modifier, puis appelle baseclient_↵ modifierclient(int). L'opérateur peut modifier le nom, le numéro de carte, ou les deux.

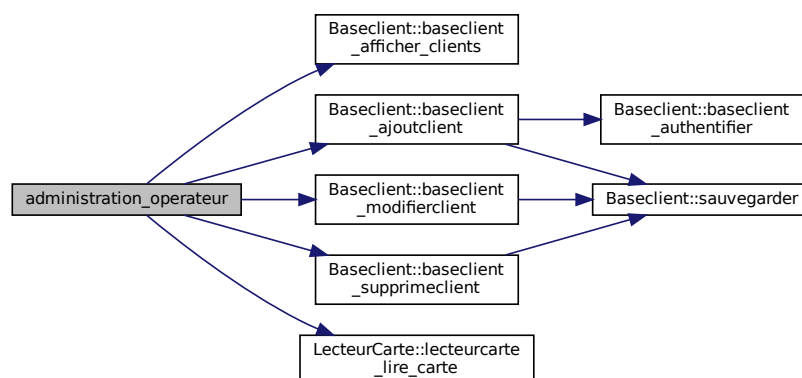
Variables internes utilisées :

- c : variable de type int pour stocker le choix de l'opération.
- num : variable de type int pour stocker le numéro de carte du client.
- nom : variable de type std::string pour stocker le nom du client (ajout uniquement).

Renvoie

void

Voici le graphe d'appel pour cette fonction :



Voici le graphe des appelants de cette fonction :



7.3.3.2 main()

```
int main ( )
```

Fonction principale du système — implémentation de la machine à états décrivant le comportement complet de la borne de recharge.

Note

Une boucle infinie est volontairement maintenue : la borne doit rester opérationnelle en permanence, sans interruption entre deux sessions de charge.

Le main orchestre l'ensemble du cycle de vie de la borne en s'appuyant sur une machine à états finie (FSM), implémentée via un switch-case sur le type énuméré `etatsystem`. À chaque tour de boucle, `etat_present` prend la valeur de `etat_suivant`, ce qui provoque la transition vers l'état suivant.

Les objets instanciés en début de fonction représentent chacun un composant physique de la borne :

- **LecteurCarte** `lecteurcarte` : lecture des cartes RFID — point d'entrée obligatoire du cycle, aucune action n'est possible sans carte valide.
- **Voyant** `voyant` : pilotage des voyants lumineux informant l'utilisateur de l'état courant (disponible, en charge, défaut).
- **Prise** `prise` : contrôle de la trappe et de la prise de charge, verrouillée durant la charge pour des raisons de sécurité.
- **Timer** `timer` : mesure du temps écoulé en secondes, utilisé pour la temporisation de 60 secondes en attente d'appui bouton.
- **Generateur_Save** `generateur_save` : pilotage du signal PWM et du contacteur AC selon la phase de charge (DC, AC_1K, AC_CL, STOP).
- **Bouton** `bouton` : lecture des boutons physiques CHARGE et STOP, ce dernier pouvant interrompre la charge depuis les états 2, 3 et 4.

Les états de la machine sont les suivants :

- **etat0** : lecture de la carte RFID et authentification du client. Si le client est reconnu, il dispose de 60 secondes pour appuyer sur le bouton CHARGE afin de démarrer le cycle.
- **etat1** : initialisation du cycle de charge — déverrouillage de la trappe et attente du branchement de la prise par le client.
- **etat2** : prise branchée — verrouillage de la trappe, passage en mode AC et ouverture du contacteur.
- **etat3** : charge active du véhicule — fermeture du contacteur et surveillance de la tension DC jusqu'en fin de charge.
- **etat4** : fin de charge — re-authentification du client requise avant de pouvoir récupérer son véhicule.
- **etat5** : récupération du véhicule — déverrouillage de la trappe, extinction de la prise et attente du débranchement.
- **etat6** : remise en disponibilité de la borne pour le client suivant.
- **etat255** : mode administration — réservé à l'opérateur pour la gestion de la base clients (ajout, suppression, modification).

Avertissement

L'état initial auquel fait référence `etat6` est différent de `etat0` dans le sens où il s'agit d'un état de réinitialisation matérielle de la borne (contacteur ouvert, PWM stoppé, voyant disponible allumé), tandis que `etat0` représente l'attente active d'un client avec lecture de carte.

Variables internes utilisées :

- **etat_present** : de type `etatsystem`, stocke l'état courant du système.
- **etat_suivant** : de type `etatsystem`, stocke l'état vers lequel transiter.
- **numero** : entier `int` stockant le numéro lu par le lecteur de carte RFID.
- **found** : entier `int` stockant le résultat de l'authentification (1 = client reconnu, 0 = échec).
- **timer_secs** : entier `int` comptabilisant les secondes écoulées durant la temporisation de 60 secondes.
- **butt_apuie** : entier `int` indiquant si le bouton CHARGE a été appuyé (0 = appuyé, 1 = non appuyé).
- **id** : entier `int` servant d'indicateur pour la re-authentification lors de la reprise du véhicule (0 = non authentifié, 1 = ok).
- **data** : entier `int` stockant le numéro saisi par le client lors de la re-authentification en `etat4` et `etat5`.

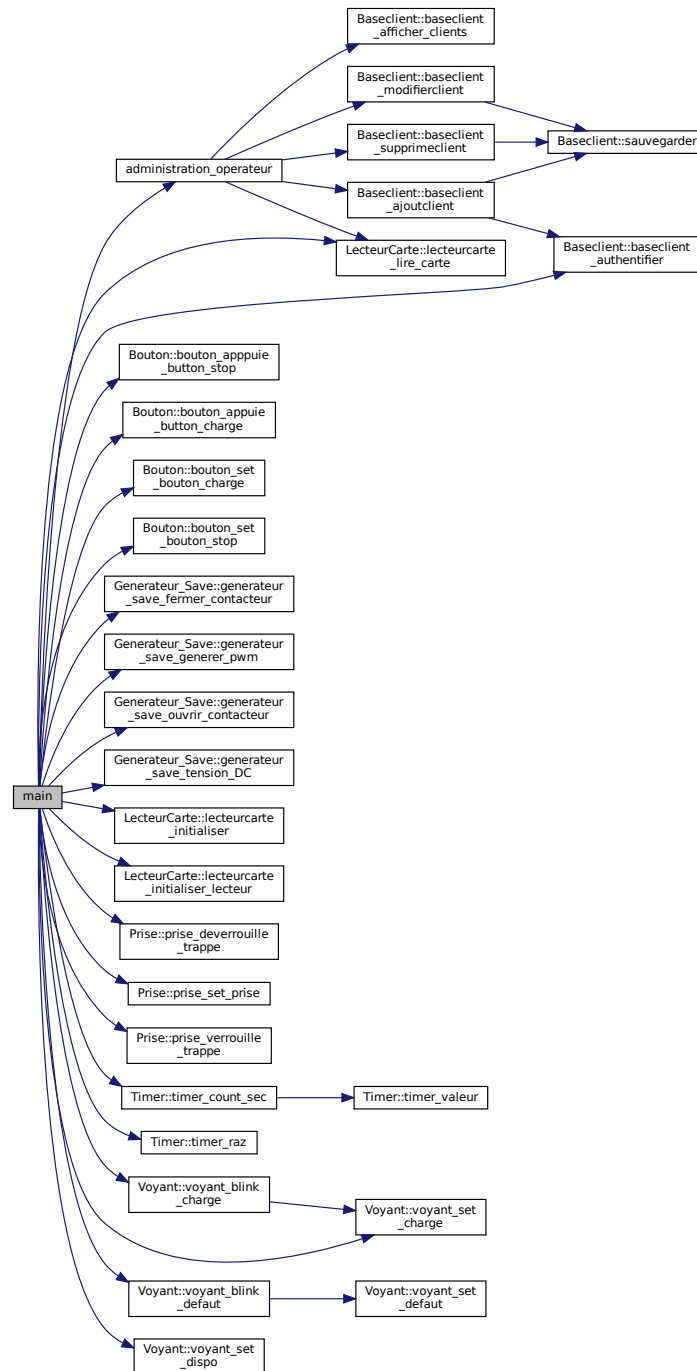
Renvoie

`int`

Valeurs retournées

0	en cas de succès d'exécution (jamais atteint en pratique, la boucle infinie étant intentionnelle).
---	--

Voici le graphe d'appel pour cette fonction :



7.3.4 Documentation des variables

7.3.4.1 baseclient

`Baseclient` baseclient

7.3.4.2 lecteurcarte

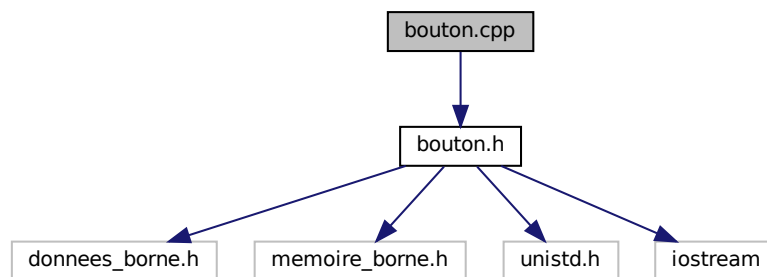
`LecteurCarte` `lecteurcarte`

7.4 Référence du fichier bouton.cpp

Implémentation des méthodes de la classe `Bouton`.

```
#include "bouton.h"
```

Graphe des dépendances par inclusion de bouton.cpp:



7.4.1 Description détaillée

Implémentation des méthodes de la classe `Bouton`.

Ce fichier contient l'implémentation des méthodes de lecture et de remise à zéro des boutons physiques de la borne. L'accès aux registres matériels se fait via un segment de mémoire partagée, obtenu à l'instanciation de l'objet grâce à la fonction `acces_memoire()`.

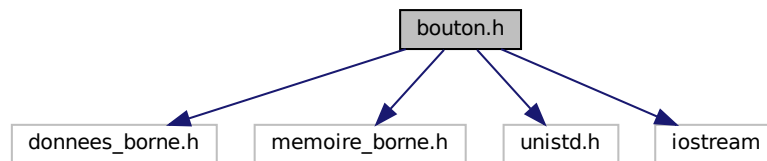
7.5 Référence du fichier bouton.h

Déclaration de la classe `Bouton` — gestion des boutons physiques de la borne.

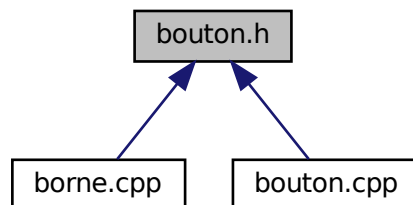
```
#include <donnees_borne.h>
#include <memoire_borne.h>
#include <unistd.h>
```

```
#include <iostream>
```

Graphe des dépendances par inclusion de bouton.h:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



Classes

- class [Bouton](#)
Gère la lecture et la remise à zéro des boutons physiques de la borne.

Macros

- #define [IOSTREAM](#)

7.5.1 Description détaillée

Déclaration de la classe [Bouton](#) — gestion des boutons physiques de la borne.

Ce fichier déclare la classe [Bouton](#) qui assure l'interface entre le logiciel et les deux boutons physiques de la borne de recharge :

- le bouton **CHARGE** : déclenché par le client pour démarrer la charge.
- le bouton **STOP** : déclenché pour interrompre la charge à tout moment.

La classe s'appuie sur la structure `entrees` définie dans `donnees_borne.h` pour accéder aux entrées matérielles de la borne via un pointeur mémoire mappé depuis `memoire_borne.h`.

7.5.2 Documentation des macros

7.5.2.1 IOSTREAM

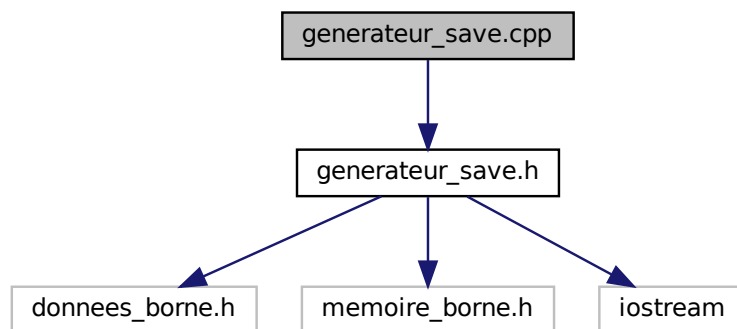
```
#define IOSTREAM
```

7.6 Référence du fichier `generateur_save.cpp`

Implémentation des méthodes de la classe [Générateur_Save](#).

```
#include "generateur_save.h"
```

Grappe des dépendances par inclusion de `generateur_save.cpp`:



7.6.1 Description détaillée

Implémentation des méthodes de la classe [Générateur_Save](#).

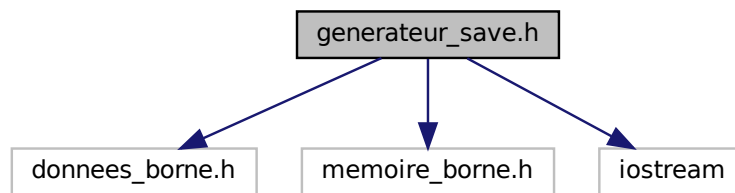
Ce fichier contient l'implémentation des méthodes de pilotage du générateur PWM, du contacteur AC et de la lecture de tension DC. Chaque méthode agit directement sur les registres matériels de la borne via le pointeur `io_g`, initialisé à la construction de l'objet par accès au segment de mémoire partagée.

7.7 Référence du fichier `generateur_save.h`

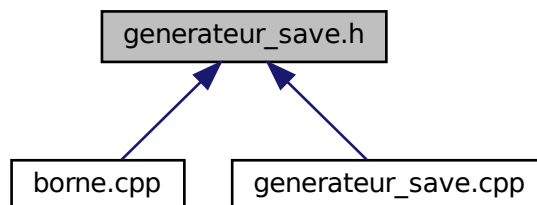
Déclaration de la classe `Generateur_Save` — pilotage du générateur de signal PWM et du contacteur AC de la borne de recharge.

```
#include <donnees_borne.h>
#include <memoire_borne.h>
#include <iostream>
```

Graphe des dépendances par inclusion de `generateur_save.h`:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



Classes

- class `Generateur_Save`
Pilote le générateur PWM, le contacteur AC et la lecture de tension DC de la borne de recharge.

Macros

- #define `IOSTREAM`

7.7.1 Description détaillée

Déclaration de la classe `Generateur_Save` — pilotage du générateur de signal PWM et du contacteur AC de la borne de recharge.

Ce fichier déclare la classe `Generateur_Save` qui assure l'interface entre le logiciel et les composants de puissance de la borne :

- le **générateur PWM** : produit les signaux nécessaires aux différentes phases de charge (DC, AC_1K, AC_2K, AC_3K, STOP).
- le **contacteur AC** : interrupteur électromécanique permettant d'ouvrir ou de fermer le circuit d'alimentation du véhicule.
- la **tension DC** : mesure de la tension côté véhicule, utilisée pour surveiller la progression et la fin de charge.

La classe s'appuie sur la structure `entrees` définie dans `donnees_borne.h`, accessible via un segment de mémoire partagée fourni par `memoire_borne.h`.

7.7.2 Documentation des macros

7.7.2.1 IOSTREAM

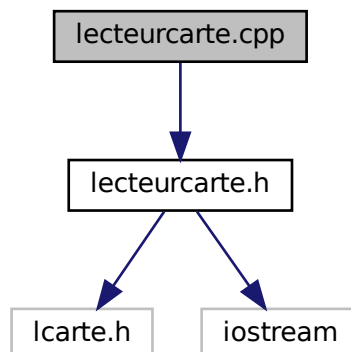
```
#define IOSTREAM
```

7.8 Référence du fichier lecteurcarte.cpp

Implémentation des méthodes de la classe `LecteurCarte`.

```
#include "lecteurcarte.h"
```

Graphe des dépendances par inclusion de `lecteurcarte.cpp`:



7.8.1 Description détaillée

Implémentation des méthodes de la classe [LecteurCarte](#).

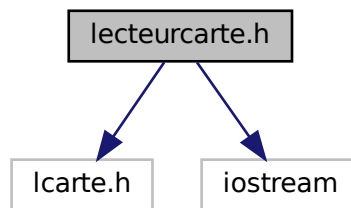
Ce fichier contient l'implémentation des méthodes de gestion du lecteur de cartes de la borne. Dans cette version, la lecture est simulée par une saisie clavier via `cin`, ce qui permet de tester le comportement de la machine à états sans matériel physique connecté.

7.9 Référence du fichier `lecteurcarte.h`

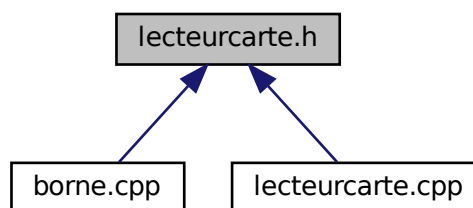
Déclaration de la classe [LecteurCarte](#) — gestion du lecteur de cartes de la borne de recharge.

```
#include <lcarte.h>
#include <iostream>
```

Graphe des dépendances par inclusion de `lecteurcarte.h`:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



Classes

- class [LecteurCarte](#)
Gère l'initialisation et la lecture du lecteur de cartes.

Macros

— #define `IOSTREAM`

7.9.1 Description détaillée

Déclaration de la classe `LecteurCarte` — gestion du lecteur de cartes de la borne de recharge.

Ce fichier déclare la classe `LecteurCarte` qui assure l'interface entre le logiciel et le lecteur de cartes physique de la borne. Elle expose trois méthodes couvrant l'ensemble du cycle d'utilisation du lecteur :

- l'**initialisation du port** : effectuée une seule fois au démarrage du système, avant la boucle principale.
- l'**initialisation de session** : effectuée à chaque passage en `etat0`, pour préparer une nouvelle lecture.
- la **lecture de carte** : retourne le numéro lu depuis la carte présentée par le client ou l'opérateur.

La classe s'appuie sur la bibliothèque `lcarte.h` qui fournit les primitives bas niveau d'accès au lecteur de cartes.

Note

Le numéro 255 est une valeur réservée retournée par le lecteur lorsqu'une carte opérateur est détectée. Il déclenche alors la transition vers `etat255` (mode administration).

7.9.2 Documentation des macros

7.9.2.1 IOSTREAM

```
#define IOSTREAM
```

7.10 Référence du fichier page2.md

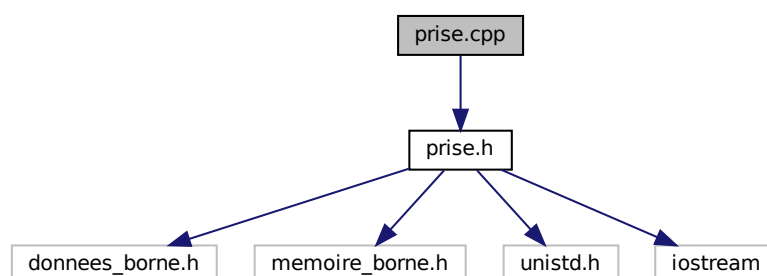
7.11 Référence du fichier page3.md

7.12 Référence du fichier prise.cpp

Implémentation des méthodes de la classe `Prise`.

```
#include "prise.h"
```

Graphe des dépendances par inclusion de `prise.cpp`:



7.12.1 Description détaillée

Implémentation des méthodes de la classe [Prise](#).

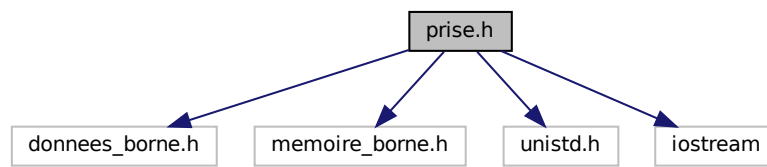
Ce fichier contient l'implémentation des méthodes de contrôle de la trappe mécanique et du voyant de la prise de charge. Chaque méthode agit directement sur les registres matériels de la borne via le pointeur `io_p`, initialisé à la construction de l'objet par accès au segment de mémoire partagée.

7.13 Référence du fichier prise.h

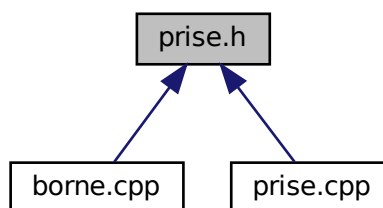
Déclaration de la classe [Prise](#) — gestion de la trappe et du voyant de la prise de charge de la borne.

```
#include <donnees_borne.h>
#include <memoire_borne.h>
#include <unistd.h>
#include <iostream>
```

Graphe des dépendances par inclusion de prise.h:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



Classes

- class [Prise](#)
Pilote la trappe mécanique et le voyant de la prise de charge.

Macros

— #define `IOSTREAM`

7.13.1 Description détaillée

Déclaration de la classe `Prise` — gestion de la trappe et du voyant de la prise de charge de la borne.

Ce fichier déclare la classe `Prise` qui assure l'interface entre le logiciel et les actionneurs physiques de la prise de charge :

- la **trappe mécanique** : verrouillée durant la charge pour sécuriser la connexion entre le câble et le véhicule, déverrouillée lors du branchement et du débranchement du câble.
- le **voyant de la prise** : indique visuellement l'état de la prise au client (disponible, en cours de charge, éteinte).

La classe s'appuie sur la structure `entrees` définie dans `donnees_borne.h`, accessible via un segment de mémoire partagée fourni par `memoire_borne.h`.

7.13.2 Documentation des macros

7.13.2.1 IOSTREAM

```
#define IOSTREAM
```

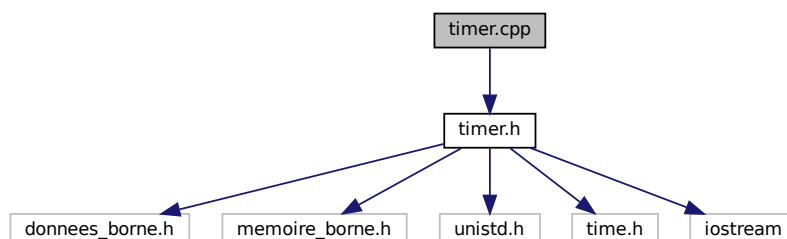
7.14 Référence du fichier README.md

7.15 Référence du fichier timer.cpp

Implémentation des méthodes de la classe `Timer`.

```
#include "timer.h"
```

Graphe des dépendances par inclusion de `timer.cpp`:



7.15.1 Description détaillée

Implémentation des méthodes de la classe [Timer](#).

Ce fichier contient l'implémentation des méthodes de temporisation de la borne. La mesure du temps repose sur deux mécanismes combinés :

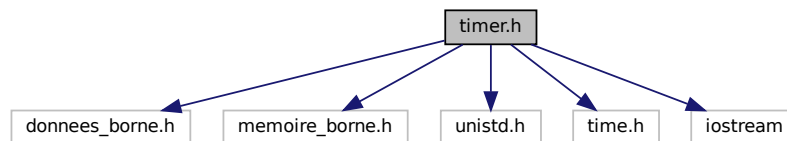
- un registre matériel `timer_sec` accessible via `io_t`, mis à jour à chaque appel à `timer_count_sec()` pour refléter le temps écoulé côté hardware.
- un horodatage logiciel via `time()` et `depart_timer`, qui sert de référence pour calculer les intervalles de temps entre deux appels.

7.16 Référence du fichier timer.h

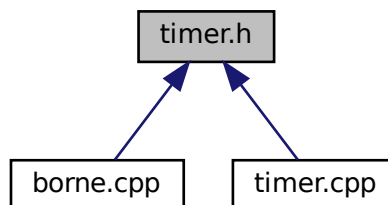
Déclaration de la classe [Timer](#) — gestion de la temporisation du système de borne de recharge.

```
#include <donnees_borne.h>
#include <memoire_borne.h>
#include <unistd.h>
#include <time.h>
#include <iostream>
```

Graphe des dépendances par inclusion de timer.h:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



Classes

- class [Timer](#)
Gère la temporisation et la mesure du temps écoulé dans la borne.

Macros

— #define `IOSTREAM`

7.16.1 Description détaillée

Déclaration de la classe `Timer` — gestion de la temporisation du système de borne de recharge.

Ce fichier déclare la classe `Timer` qui fournit les mécanismes de mesure du temps écoulé utilisés dans la machine à états de la borne. Elle est principalement sollicitée dans `etat0`, où le client dispose de 60 secondes pour appuyer sur le bouton CHARGE après authentification.

La classe combine deux approches complémentaires :

- un **accès aux registres matériels** via la mémoire partagée, pour lire la valeur du timer hardware de la borne.
- un **horodatage logiciel** via `time_t`, pour mesurer le temps écoulé depuis la dernière remise à zéro.

La classe s'appuie sur la structure `entrees` définie dans `donnees_borne.h`, accessible via un segment de mémoire partagée fourni par `memoire_borne.h`, ainsi que sur la bibliothèque standard `time.h` pour la gestion de l'horodatage logiciel.

7.16.2 Documentation des macros

7.16.2.1 IOSTREAM

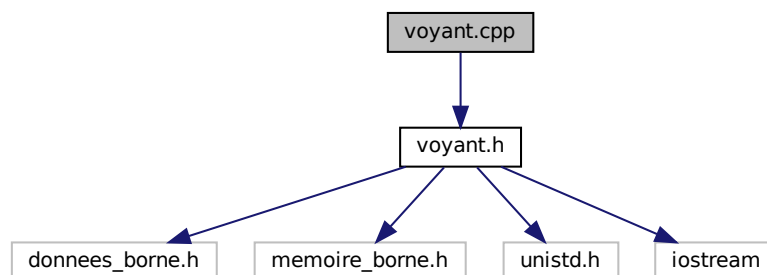
```
#define IOSTREAM
```

7.17 Référence du fichier voyant.cpp

Implémentation des méthodes de la classe `Voyant`.

```
#include "voyant.h"
```

Graphe des dépendances par inclusion de `voyant.cpp`:



7.17.1 Description détaillée

Implémentation des méthodes de la classe [Voyant](#).

Ce fichier contient l'implémentation des méthodes de pilotage des voyants lumineux de la borne. Chaque méthode agit directement sur les champs de la structure `entrees` via le pointeur `io`, initialisé à la construction de l'objet par accès au segment de mémoire partagée.

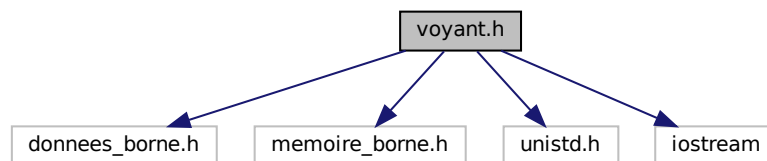
Les méthodes `voyant_blink_charge()` et `voyant_blink_default()` sont les seules méthodes bloquantes de la classe — elles immobilisent le fil d'exécution pendant 8 secondes via des appels successifs à `sleep(1)`.

7.18 Référence du fichier voyant.h

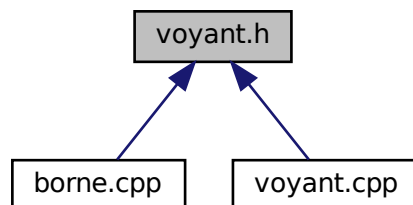
Déclaration de la classe [Voyant](#) — gestion des voyants lumineux de la borne de recharge.

```
#include <donnees_borne.h>
#include <memoire_borne.h>
#include <unistd.h>
#include <iostream>
```

Graphe des dépendances par inclusion de voyant.h:



Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



Classes

- class [Voyant](#)
Pilote l'ensemble des voyants lumineux de la borne de recharge.

Macros

— #define `IOSTREAM`

7.18.1 Description détaillée

Déclaration de la classe `Voyant` — gestion des voyants lumineux de la borne de recharge.

Ce fichier déclare la classe `Voyant` qui assure l'interface entre le logiciel et les voyants lumineux physiques de la borne. Ces voyants constituent le retour visuel principal pour le client et reflètent à tout moment l'état courant du système :

- le **voyant disponibilité** : indique si la borne est libre ou occupée.
- le **voyant charge** : signale l'état du cycle de charge (en attente, en cours, terminé).
- le **voyant défaut** : s'allume ou clignote en cas d'échec d'authentification ou d'anomalie.
- le **voyant prise** : indique l'état de la prise de charge.
- le **voyant trappe** : indique l'état du verrou de la trappe.

La classe s'appuie sur la structure `entrees` définie dans `donnees_borne.h`, accessible via un segment de mémoire partagée fourni par `memoire_borne.h`.

Note

Les méthodes `voyant_blink_charge()` et `voyant_blink_defaut()` produisent un clignotement pendant 8 secondes. Elles sont bloquantes durant toute la durée du clignotement.

7.18.2 Documentation des macros

7.18.2.1 IOSTREAM

```
#define IOSTREAM
```


Index

- administration_operateur
 - borne.cpp, 53
- Baseclient, 11
 - Baseclient, 12
 - baseclient_afficher_clients, 12
 - baseclient_ajoutclient, 13
 - baseclient_authentifier, 14
 - baseclient_modifierclient, 15
 - baseclient_supprimeclient, 16
 - charger, 17
 - clients, 18
 - fichier, 18
 - sauvegarder, 17
- baseclient
 - borne.cpp, 56
- baseclient.cpp, 49
- baseclient.h, 50
 - IOSTREAM, 51
- baseclient_afficher_clients
 - Baseclient, 12
- baseclient_ajoutclient
 - Baseclient, 13
- baseclient_authentifier
 - Baseclient, 14
- baseclient_modifierclient
 - Baseclient, 15
- baseclient_supprimeclient
 - Baseclient, 16
- borne.cpp, 51
 - administration_operateur, 53
 - baseclient, 56
 - etat0, 52
 - etat1, 52
 - etat2, 52
 - etat255, 52
 - etat3, 52
 - etat4, 52
 - etat5, 52
 - etat6, 52
 - etatsystem, 52
 - lecteurcarte, 57
 - main, 54
- Bouton, 19
 - Bouton, 19
 - bouton_appuie_button_stop, 20
 - bouton_appuie_button_charge, 20
 - bouton_set_bouton_charge, 21
 - bouton_set_bouton_stop, 22
 - io_b, 23
- bouton.cpp, 57
- bouton.h, 57
 - IOSTREAM, 59
- bouton_appuie_button_stop
 - Bouton, 20
- bouton_appuie_button_charge
 - Bouton, 20
- bouton_set_bouton_charge
 - Bouton, 21
- bouton_set_bouton_stop
 - Bouton, 22
- charger
 - Baseclient, 17
- Client, 23
 - nom, 24
 - numero, 24
- clients
 - Baseclient, 18
- depart_timer
 - Timer, 40
- etat0
 - borne.cpp, 52
- etat1
 - borne.cpp, 52
- etat2
 - borne.cpp, 52
- etat255
 - borne.cpp, 52
- etat3
 - borne.cpp, 52
- etat4
 - borne.cpp, 52
- etat5
 - borne.cpp, 52
- etat6
 - borne.cpp, 52
- etatsystem
 - borne.cpp, 52
- fichier
 - Baseclient, 18
- Generateur_Save, 24
 - Generateur_Save, 25
 - generateur_save_fermer_contacteur, 25
 - generateur_save_generer_pwm, 26
 - generateur_save_ouvrir_contacteur, 27
 - generateur_save_tension_DC, 28

- io_g, 29
- generateur_save.cpp, 59
- generateur_save.h, 60
- Iostream, 61
- generateur_save_fermer_contacteur
 - Generateur_Save, 25
- generateur_save_generer_pwm
 - Generateur_Save, 26
- generateur_save_ouvrir_contacteur
 - Generateur_Save, 27
- generateur_save_tension_DC
 - Generateur_Save, 28
- io
 - Voyant, 48
- io_b
 - Bouton, 23
- io_g
 - Generateur_Save, 29
- io_p
 - Prise, 36
- io_t
 - Timer, 40
- Iostream
 - baseclient.h, 51
 - bouton.h, 59
 - generateur_save.h, 61
 - lecteurcarte.h, 63
 - prise.h, 65
 - timer.h, 67
 - voyant.h, 69
- LecteurCarte, 29
 - lecteurcarte_initialiser, 30
 - lecteurcarte_initialiser_lecteur, 30
 - lecteurcarte_lire_carte, 31
- lecteurcarte
 - borne.cpp, 57
- lecteurcarte.cpp, 61
- lecteurcarte.h, 62
- Iostream, 63
- lecteurcarte_initialiser
 - LecteurCarte, 30
- lecteurcarte_initialiser_lecteur
 - LecteurCarte, 30
- lecteurcarte_lire_carte
 - LecteurCarte, 31
- main
 - borne.cpp, 54
- nom
 - Client, 24
- numero
 - Client, 24
- page2.md, 63
- page3.md, 63
- Prise, 32
 - io_p, 36
 - Prise, 33
 - prise_deverrouille_trappe, 33
 - prise_set_prise, 34
 - prise_verrouille_trappe, 35
- prise.cpp, 63
- prise.h, 64
 - Iostream, 65
- prise_deverrouille_trappe
 - Prise, 33
- prise_set_prise
 - Prise, 34
- prise_verrouille_trappe
 - Prise, 35
- README.md, 65
- sauvegarder
 - Baseclient, 17
- Timer, 36
 - depart_timer, 40
 - io_t, 40
 - Timer, 37
 - timer_count_sec, 37
 - timer_raz, 38
 - timer_valeur, 39
- timer.cpp, 65
- timer.h, 66
 - Iostream, 67
- timer_count_sec
 - Timer, 37
- timer_raz
 - Timer, 38
- timer_valeur
 - Timer, 39
- Voyant, 41
 - io, 48
 - Voyant, 41
 - voyant_blink_charge, 42
 - voyant_blink_default, 43
 - voyant_set_charge, 44
 - voyant_set_default, 45
 - voyant_set_dispo, 46
 - voyant_set_prise, 47
 - voyant_set_trappe, 47
- voyant.cpp, 67
- voyant.h, 68
 - Iostream, 69
- voyant_blink_charge
 - Voyant, 42
- voyant_blink_default
 - Voyant, 43
- voyant_set_charge
 - Voyant, 44
- voyant_set_default
 - Voyant, 45
- voyant_set_dispo

Voyant, [46](#)
voyant_set_prise
Voyant, [47](#)
voyant_set_trappe
Voyant, [47](#)